

capability is a name for a resource managed by a subsystem (or, in a few cases, by the kernel). It consists of the 64-bit UI of a port belonging to that subsystem and a 64-bit key assigned by the subsystem, as shown in Fig. 9-3.

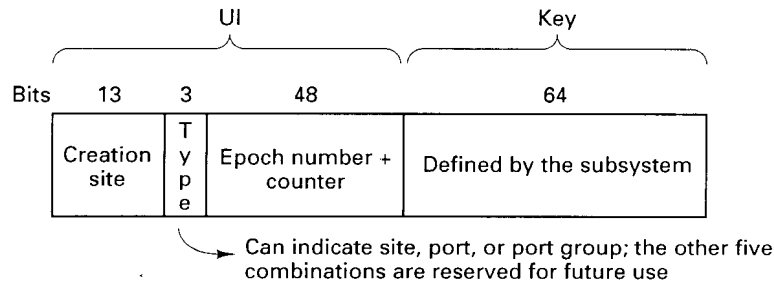


Fig. 9-3. A capability in Chorus.

When a subsystem creates an object, such as a file, it returns to the caller the capability for the object. From the capability, that process, or any subsequent process acquiring the capability, can find the UI of a port to which messages can be sent to request operations on the object. The 64-bit key must be included in such messages to tell the subsystem which of its many objects is being referenced. Included in the 64 bits is an index into the subsystem's tables, to identify the object. Other bits are randomly chosen to make it difficult to guess valid capabilities. Like UIs, capabilities may be freely passed in messages and files. This naming scheme was taken from Amoeba (Tanenbaum et al., 1986).

Memory management in Chorus is based on two concepts, regions, which were described above, and segments. A **segment** is a linear sequence of bytes identified by a capability. When a segment is mapped onto a region, the bytes of the segment are accessible to the threads of the region's process just by reading and writing addresses in the region. Programs, files, and other forms of data are stored as segments in Chorus and can be mapped onto regions. A segment can also be read and written by system calls, even when it is not mapped onto a region. An example address space is shown in Fig. 9-4.

9.1.5. Kernel Structure

Having described the main abstractions provided by the Chorus kernel, let us now briefly examine how the kernel is structured internally. The kernel consists of four pieces, as illustrated in Fig. 9-5. At the bottom is the **supervisor**, which manages the raw hardware and catches traps, exceptions, interrupts, and other hardware details, and handles context switching. It is written partly in assembler and has to be redone when Chorus is ported to new hardware.

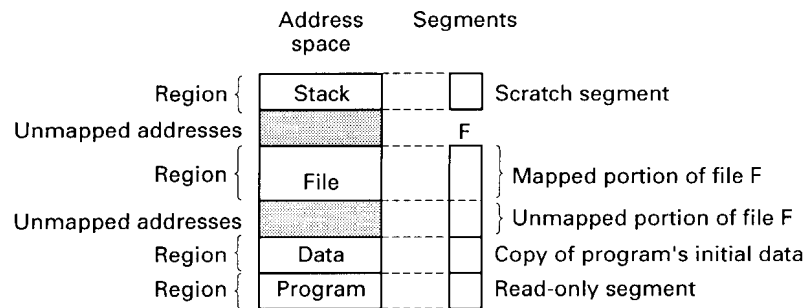


Fig. 9-4. An address space with four mapped regions.

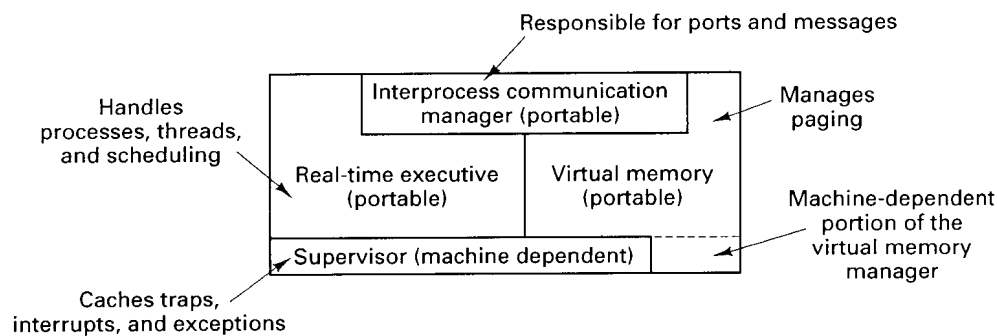


Fig. 9-5. Structure of the Chorus kernel.

Next comes the **virtual memory manager**, which handles the low-level part of the paging system. The largest piece of it deals with managing page caches and other logical concepts, and is machine independent. A small part, however, has to know how to load and store the MMU registers. This part is machine dependent and has to be modified when Chorus is ported to a new computer.

The two parts of the virtual memory manager together do not do all the work of managing the paging system. A third part, the mapper, is outside the kernel and does the high-level part. The communication protocol between the virtual memory manager and the mapper is well defined, so users can provide their own specialized mappers.

The third part of the kernel is the **real-time executive**. It is responsible for managing processes, threads, and scheduling. It also takes care of arranging for synchronization between threads for mutual exclusion and other purposes.

Finally, we have the **interprocess communication manager**, which handles UIs, ports, and the sending of messages in a transparent way. It makes use of the services of the real-time executive and virtual memory manager to do its

work. It is completely portable and does not have to be changed at all when Chorus is moved to a new platform. The four parts of the kernel are constructed in a modular way, so changes to one usually do not affect any of the others.

9.1.6. The UNIX Subsystem

Since Chorus is now a commercial product, it must give the masses what they want, and what they want (at least in the high-end workstation world) is compatibility with UNIX. Chorus accomplishes this goal by providing a standard subsystem, called **MiX**, that is compatible with System V. The compatibility is both at the source level (i.e., UNIX source programs can be compiled and run on Chorus) and at the binary level (i.e., executable programs compiled on a true UNIX system for the same architecture run without modification on Chorus). An earlier version of MiX (3.2) was compatible with 4.2 BSD, but we will not discuss that version further in this chapter.

MiX is also compatible with UNIX in other ways. For example, the file system is compatible, so Chorus can read a UNIX disk. Furthermore, the Chorus device drivers are interface compatible with the UNIX ones, so if UNIX device drivers exist for a device machine, they can be ported to Chorus with relatively little work.

The implementation of the MiX subsystem is more modular than UNIX. It consists of four processes, one for process management, one for file management, one for device management, and one for streams and interprocess communication. These processes do not share any variables or other memory, and communicate exclusively by remote procedure call. Later in this chapter we will describe in detail how they work.

9.1.7. The Object-Oriented Subsystem

As a research experiment, a second subsystem has been implemented, this one for object-oriented programming. It consists of three layers. The bottom layer does object management in a generic way and is effectively a microkernel for object-oriented systems. The middle layer provides a general runtime system. The top layer is the language runtime system. This subsystem, called COOL, will also be discussed later in this chapter.

9.2. PROCESS MANAGEMENT IN CHORUS

In this section we will describe how processes and threads work in Chorus, how exceptions are handled, and how scheduling is done. We will conclude by describing briefly some of the major process management kernel calls available.

9.2.1. Processes

A process in Chorus is a collection of active and passive elements that work together to perform some computation. The active elements are the threads. The passive elements are an address space (containing some regions) and a collection of ports (for sending and receiving messages). A process with one thread is like a traditional UNIX process. A process with no threads cannot do anything useful, and normally exists only for a very short interval while a process is being created.

Three kinds of processes exist, differing in the amount of privilege and trust they have, as listed in Fig. 9-6. Privilege refers to the ability to execute I/O and other protected instructions. Trust means that the process is allowed to call the kernel directly.

Type	Trust	Privilege	Mode	Space
User	Untrusted	Unprivileged	User	User
System	Trusted	Unprivileged	User	User
Kernel	Trusted	Privileged	Kernel	Kernel

Fig. 9-6. The three kinds of processes in Chorus.

Kernel processes are the most powerful. They run in kernel mode and all share the same address space with each other and with the microkernel. They can be loaded and unloaded during execution, but other than that, can be thought of as extensions to the microkernel itself. Kernel processes can communicate with each other using a special lightweight RPC that is not available to other processes.

Each system process runs in its own address space. System processes are unprivileged (i.e., run in user mode), and thus cannot execute I/O and other protected instructions directly. However, the kernel trusts them to make kernel calls, so system processes can obtain kernel services directly, without any intermediary.

User processes are untrusted and unprivileged. They cannot perform I/O directly, and cannot even call the kernel, except for those calls that their subsystem has decided to make on their behalf. Each user process has two parts: the regular user part and a system part that is invoked after a trap. This arrangement is similar to the way that UNIX works.

Every process (and port) has a **protection identifier** associated with it. If the process forks, its children inherit the same protection identifier. This identifier is just a bit string, and does not have any semantics associated with it that the kernel knows about. Protection identifiers provide a mechanism which can be used for authentication. For example, the UNIX subsystem could assign a

UID (user identifier) with each process and use the Chorus protection identifiers to implement the UIDs.

9.2.2. Threads

Every active process in Chorus has one or more threads that execute code. Each thread has its own private context (i.e., stack, program counter, and registers), which is saved when the thread blocks waiting for some event and is restored when the thread is later resumed. A thread is tied to the process in which it was created, and cannot be moved to another process.

Chorus threads are known to the kernel and scheduled by the kernel, so creating and destroying them requires making kernel calls. An advantage of having kernel threads (as opposed to a threads package that runs entirely in user space, without kernel knowledge), is that when one thread blocks waiting for some event (e.g., a message arrival), the kernel can schedule other threads. Another advantage is the ability to run different threads on different CPUs when a multiprocessor is available. The disadvantage of kernel threads is the extra overhead required to manage them. Of course, users are still free to implement a user-level threads package inside a single kernel thread.

Threads communicate with one another by sending and receiving messages. It does not matter if the sender and receiver are in the same process or are on different machines. The semantics of communication are identical in all cases. If two threads are in the same process, they can also communicate using shared memory, but then the system cannot later be reconfigured to run with threads in different processes.

The following states are distinguished, but they are not mutually exclusive:

1. ACTIVE — The thread is logically able to run.
2. SUSPENDED — The thread has been intentionally suspended.
3. STOPPED — The thread's process has been suspended.
4. WAITING — The thread is waiting for some event to happen.

A thread in the ACTIVE state is either currently running or waiting its turn for a free CPU. In both cases it is logically unblocked and able to run. A thread in the SUSPENDED state has been suspended by another thread (or itself) that issued a kernel call asking the kernel to suspend the thread. Similarly, when a kernel call is made to stop a process, all the threads in the ACTIVE state are put in the STOPPED state until the process is released. Finally, when a thread performs a blocking operation that cannot be completed immediately, the thread is put in WAITING state until the event occurs.

A thread can be in more than one state at the same time. For example, a

thread in SUSPENDED state can later also enter the STOPPED state as well if its process is suspended. Conceptually, each thread has three independent bits associated with it, one each for SUSPENDED, STOPPED, and WAITING. Only when all three bits are zero can the thread run.

Threads run in the mode and address space corresponding to their process. In other words, the threads of a kernel process run in kernel mode, and the threads of a user process run in user.

The kernel provides two synchronization mechanisms that threads can use. The first is the traditional (counting) semaphore, with operations UP (or V) and DOWN (or P). These operations are always implemented by kernel calls, so they are expensive. The second mechanism is the mutex, which is essentially a semaphore whose values are restricted to 0 and 1. Mutexes are used only for mutual exclusion. They have the advantage that operations that do not cause the caller to block can be carried out entirely in the caller's space, saving the overhead of a kernel call.

A problem that occurs in every thread-based system is how to manage the data private to each thread, such as its stack. Chorus solves this problem by assigning two special **software registers** to each thread. One of them holds a pointer to the thread's private data when it is in user mode. The other holds a pointer to the private data when the thread has trapped to the kernel and is executing a kernel call. Both registers are part of the thread's state, and are saved and restored along with the hardware registers when a thread is stopped or started. By indexing off these registers, a thread can access data that (by convention) are not available to other threads in the same process.

9.2.3. Scheduling

CPU scheduling is done using priorities on a per-thread basis. Each process has a priority and each thread has a relative priority within its process. The absolute priority of a thread is the sum of its process' priority and its own relative priority. The kernel keeps track of the priority of each thread in ACTIVE state and runs the one with the highest absolute priority. On a multiprocessor with k CPUs, the k highest-priority threads are run.

However, to accommodate real-time processes, an additional feature has been added to the algorithm. A distinction is made between threads whose priority is above a certain level and threads whose priority is below it. High-priority threads, such as *A* and *B* in Fig. 9-7(a), are not timesliced. Once such a thread starts running, it continues to run until either it voluntarily releases its CPU (e.g., by blocking on a semaphore), or an even higher priority thread moves into the ACTIVE state as a result of I/O completing or some other event happening. In particular, it is not stopped just because it has run for a long time.

In contrast, in Fig. 9-7(b), thread *C* will be run, but after it has consumed

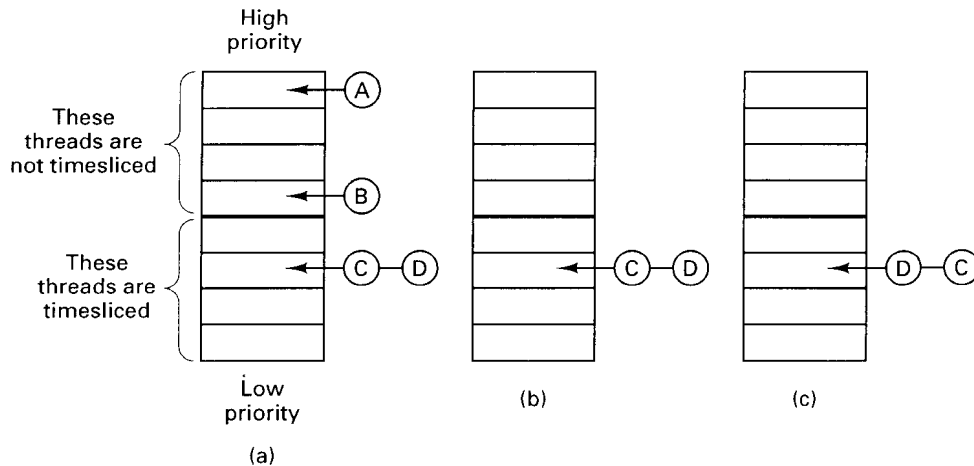


Fig. 9-7. (a) Thread *A* will be run until it is finished or it blocks. (b)–(c) Threads *C* and *D* will alternate, in round robin mode.

one quantum of CPU time, it will be put on the end of the queue for its priority, and thread *D* will be given one quantum. In the absence of competition for the CPU, they will alternate indefinitely.

This mechanism provides enough generality for most real-time applications. System calls are available for changing process and thread priorities, so applications can tell the system which threads are most important and which are least important. Additional scheduling algorithms are available to support System V real-time and system processes.

9.2.4. Traps, Exceptions, and Interrupts

The Chorus software distinguishes between three kinds of entries into the kernel. **Traps** are intentional calls to the kernel or a subsystem to invoke services. Programs cause traps by calling a system call library procedure. The system supports two ways of handling traps. In the first way, all traps for a particular trap vector go to a single kernel thread that has previously announced its willingness to handle that vector. In the second way, each trap vector is tied to an array of kernel threads, with the Chorus supervisor using the contents of a certain register to index into the array to pick a thread. The latter mechanism allows all system calls to use the same trap vector, with the system call number going into the register used to select a handler.

Exceptions are unexpected events that are caused by accident, such as the divide-by-zero exception, floating-point overflow, or a page fault. It is possible

to arrange for a kernel thread to be invoked to handle the exception. If the handler can complete the processing, it returns a special code and the exception handling is finished. Otherwise (or if no kernel handler is assigned), the kernel suspends the thread that caused the exception and sends a message to a special exception-handling port associated with the thread's process. Normally, some other thread will be waiting for a message on this port and will take whatever action the process requires. If no exception port exists, the faulting thread is killed.

Interrupts are caused by asynchronous events, such as clock ticks or the completion of an I/O request. They are not necessarily related to anything the current thread is doing, so it is not possible to let that thread's process handle them. Instead, it is possible to arrange in advance that when an interrupt occurs on a certain interrupt vector (i.e., a specific device), a new kernel thread will be created spontaneously to process it. If a second interrupt occurs on the same vector before the first one has terminated, a second thread is created, and so on. All I/O interrupts except the clock are handled this way. The clock is fielded by the supervisor itself, but it can be set up to notify a user thread if desired. Interrupt threads can invoke only a limited set of kernel services because the system is in an unknown state when they are started. All they can do are operations on semaphores and mutexes, or send minimessages to special miniports.

9.2.5. Kernel Calls for Process Management

The best way to find out what a kernel or operating system really does is to examine its interface, that is, the system calls it provides to its users. In this section we will look at the most important Chorus kernel calls available to system processes. Calls of less importance and protected calls available only to kernel threads will be omitted.

Call	Description
<code>actorCreate</code>	Create a new process
<code>actorDelete</code>	Remove a process
<code>actorStop</code>	Stop a process, put its threads in STOPPED state
<code>actorStart</code>	Restart a process from STOPPED state
<code>actorPriority</code>	Get or set a process' priority
<code>actorExcept</code>	Get or set the port used for exeception handling

Fig. 9-8. Selected process calls supported by the Chorus kernel.

Let us start with the process calls, listed in Fig. 9-8. *ActorCreate* creates a new process and returns that process' capability to the caller. The new process inherits the priority, protection identifier, and exception port of the parent

process. Parameters specify whether the new process is to be a user, system, or kernel process, and tell what state it is to start in. Just after creation, the new process is empty, with no threads and no regions and only one port, the default port. Note that *actorCreate* represents a major orthographic advance over UNIX: “Create” is spelled with an “e” at the end.

The *actorDelete* call kills a process. The process to be killed is specified by a capability passed as a parameter. *ActorStop* freezes a process, putting all of its threads into STOPPED state. The threads can only run again when an *actorStart* call is made. A process may stop itself. These calls are typically used for debugging. For example, if a thread hits a breakpoint, the debugger can use *actorStop* to stop the process’ other threads.

The *actorPriority* call allows a process to read the priority of another process, and optionally, to reset it to a new value. Although Chorus is generally location transparent, it is not perfect. Some calls, including this one, work only when the target process is on the caller’s machine. In other words, it is not possible to get or reset the priority of a distant process.

ActorExcept is used to get or change the exception port for the caller or some other process for which the caller has a capability. It can also be used to remove the exception port, in which case if an exception has to be sent to the process, the process is killed instead.

The next group of kernel calls relate to threads, and are shown in Fig. 9-9. *ThreadCreate* and *threadDelete* create and delete threads in some process (not necessarily the caller’s), respectively. Parameters to *threadCreate* specify the privilege level, initial status, priority, entry point, and stack pointer.

Call	Description
<i>threadCreate</i>	Create a new thread
<i>threadDelete</i>	Delete a thread
<i>threadSuspend</i>	Suspend a thread
<i>threadResume</i>	Restart a suspended thread
<i>threadPriority</i>	Get or set a thread's priority
<i>threadLoad</i>	Get a thread's context pointer
<i>threadStore</i>	Set a thread's context pointer
<i>threadContext</i>	Get or set a thread's execution context

Fig. 9-9. Selected thread calls supported by the Chorus kernel.

ThreadSuspend and *threadResume* stop and then restart threads in the target process. *ThreadPriority* returns the target thread’s current relative priority, and optionally resets it to a value given as a parameter.

Our last three calls are used to manage a thread’s private context. The *threadLoad* and *threadStore* calls load and set the current software context

register, respectively. This register points to the thread's context, including its private variables. The *threadContext* call optionally copies the thread's old context to a buffer, and optionally sets the new context from another buffer.

The synchronization operations are given in Fig. 9-10. Calls are provided for initializing, acquiring, and releasing both mutexes and semaphores. These all work in the usual way.

Call	Description
<code>mutexInit</code>	Initialize a mutex
<code>mutexGet</code>	Try to acquire a mutex
<code>mutexRel</code>	Release a mutex
<code>semInit</code>	Initialize a semaphore
<code>semP</code>	Do a DOWN on a semaphore
<code>semV</code>	Do an UP on a semaphore

Fig. 9-10. Selected synchronization calls supported by the Chorus kernel.

9.3. MEMORY MANAGEMENT IN CHORUS

Memory management in Chorus borrows many ideas from Mach. However, it also contains some ideas not present in Mach. In this section we will describe the basic concepts and how they are used.

9.3.1. Regions and Segments

The main concepts behind memory management in Chorus are regions and segments. A **region** is a contiguous range of virtual address, for example, 1024 to 6143. In theory, a region can begin at any virtual address and end at any virtual address, but to do anything useful, a region should be page aligned and have a length equal to some whole number of pages. All bytes in a region must have the same protection characteristics (e.g., read-only). Regions are a property of processes, and all the threads in a process see the same regions. Two regions in the same process may not overlap.

A **segment** is a contiguous collection of bytes named and protected by a capability. Files and swap areas on disk are the most common kinds of segments. Segments can be read and written using system calls that provide the segment's capability, the offset, the number of bytes, the buffer, and the transfer direction. These calls are used for doing traditional I/O operations on files.

However, another possibility is mapping segments onto regions, as shown in Fig. 9-4. It is not necessary that a segment be exactly the size of its region. If

the segment is larger than the region, only a portion of the segment will be visible in the address space, although which portion is visible can be changed by remapping it. If the segment is smaller than the region, the result of reading an unmapped address is up to the mapper. For example, it can raise an exception, return 0, or extend the segment, as it wishes.

Mapped segments are usually demand paged (unless this feature is disabled, for example, for real-time programs). When a process first references a newly mapped segment, a page fault occurs and the segment page corresponding to the address referenced is brought in and the faulting instruction restarted. In this way, ordinary virtual memory can be implemented, and in addition, a process can make one or more files visible in its virtual address space, so it can access them directly instead of having to read or write them using system calls.

The kernel supports special I/O segments for accessing the machine's I/O registers on machines with memory-mapped device registers. Using these segments, kernel threads can perform I/O by reading and writing memory directly.

9.3.2. Mappers

Chorus supports Mach-style external pagers, which are called **mappers**. Each mapper controls one or more segments that are mapped onto regions. A segment can be mapped into multiple regions, even in different address spaces at the same time, as shown in Fig. 9-11. Here segments *S1* and *S2* are both mapped into processes *A* and *B*, on the same machine but at different addresses. If process *A* writes address *A1* it changes the first word of *S1*. If process *B* later reads *B1* it also gets the value *A* wrote. Furthermore, if *S1* is a file, when both processes terminate, the change will be made in the file on disk.

The virtual memory manager in each kernel maintains a page cache and keeps track of which page belongs to which segment. Pages in the local cache can belong to named segments, such as files, or to nameless segments, such as swap areas. The kernel keeps track of which pages are clean and which are dirty. It may discard clean pages at will, but must return dirty pages to the appropriate mappers to reclaim their space.

A protocol between the kernel and mapper determines the flow of pages in both directions. When a page fault occurs, the kernel checks to see if the needed page is cached. If it is not, the kernel sends a message to the mapper controlling the page's segment asking for the page (and possibly adjacent pages as well). The faulting thread is then suspended until the page arrives.

When the mapper gets the request, it checks to see if the needed page is in its own cache (in its own address space). If not, it sends a message to the thread managing the disk to perform I/O and fetch the page. When the page arrives (or if it was already present), the mapper notifies the kernel, which then accepts the page, adjusts the MMU page tables, and resumes the faulting thread.

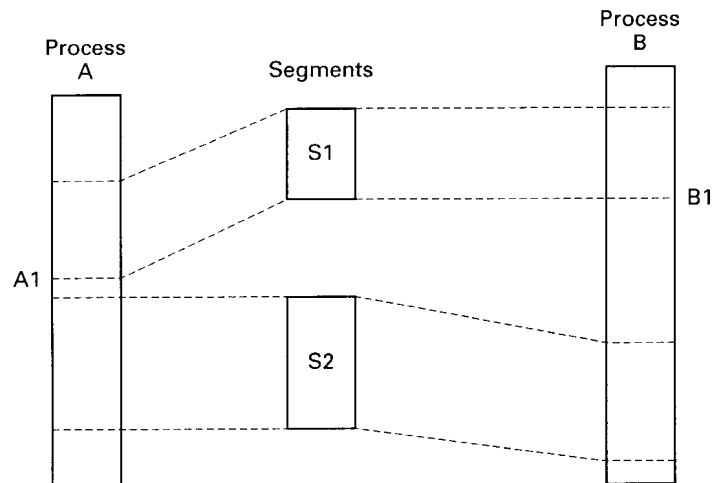


Fig. 9-11. Segments can be mapped into multiple address spaces at the same time.

The mapper can also take the initiative and ask the kernel to return dirty pages to it. When the kernel returns them, the mapper can keep some of them in its own cache and write others to disk. Various calls are provided to allow the mapper to specify which pages it wants back. Most mappers do not keep track of how many page frames their segments are occupying since the kernel is free to discard clean pages and will return dirty pages to their mappers when space gets tight.

The same caching and management mechanism is used for pages that are part of mapped segments as for pages that are read and written using explicit segment I/O commands. This approach guarantees that if one process modifies a mapped page by writing on it, and immediately thereafter another process tries to read the file it is part of, the second process will get the new data, since only one copy of the page exists in memory.

9.3.3. Distributed Shared Memory

Chorus supports paged distributed shared memory in the style of IVY, as discussed in Chap. 6. It uses a dynamic decentralized algorithm, meaning that different managers keep track of different pages, and the manager for a page changes as the page moves around the system (for writable pages).

The unit of sharing between multiple machines is the segment. Segments are split up into fragments of one or more pages. At any instant, each fragment is either read-only, and potentially present on multiple machines, or read/write, and present only on one machine.

9.3.4. Kernel Calls for Memory Management

Memory management in Chorus supports 26 different system calls plus several upcalls from the kernel to the mappers. In this section we will briefly describe just the more important ones. The calls we will describe relate to region management (*rgn* prefix), segment management (*sg* prefix), and upcalls to the mappers (*Mp* prefix—not *mp*, which is used for miniport calls, described later). The calls not described here relate to managing local caches (*lc* prefix) and virtual memory (*vm* prefix).

A selection of the region management calls is given in Fig. 9-12. *RgnAllocate* specifies a region by giving the capability for a process, a starting address, a size, and various options. If there are no conflicts with other regions and no other problems, the region is created. The options include initializing the region to zero bytes, setting its read/write/executable bits, and wiring it down so it is not paged. *RgnFree* returns a previously allocated region so its portion of the address space is free for allocation to another region or regions.

Call	Description
<i>rgnAllocate</i>	Allocate a memory region and set its properties
<i>rgnFree</i>	Release a previously allocated region
<i>rgnInit</i>	Allocate a region and fill it from a given segment
<i>rgnSetInherit</i>	Set the inheritance properties of a region
<i>rgnSetPaging</i>	Set the paging properties of a region
<i>rgnSetProtect</i>	Set the protection options of a region
<i>rgnStat</i>	Get the statistics associated with a region

Fig. 9-12. Selected calls supported by the Chorus kernel for managing regions.

RgnInit is similar to *rgnAllocate* except that after the region is allocated, it is also filled in from a segment whose capability is a parameter to the call. Several other calls that are similar to *rgnInit* are also present, filling the region in different ways.

The next three calls change the properties of an existing region in various ways. *RgnSetInherit* relates to the possibility that a region might later be copied, and specifies whether the copy is to get its own pages or to share the original region's pages. *RgnSetPaging* is used primarily by processes that are interested in providing real-time response and cannot tolerate page faults or being swapped out. It also tells what to do if the process is trying to allocate a nonswappable nonpageable region that is wired down, but there is insufficient memory available. *RgnSetProtect* changes the read/write/execute bits associated with a region and can also make a region accessible only to the kernel. Finally, *rgnStat* returns to the caller the size of a region and other information.

The segment I/O calls are shown in Fig. 9-13. *sgRead* and *sgWrite* are the basic I/O calls that allows a process to read and write a segment. The segment is specified by a descriptor. The offset and number of bytes are also provided, as is the buffer address to copy to or from. *SgStat* allows the caller, typically a mapper, to ask for statistical information about a page cache. *SgFlush* (and several related calls) allow mappers to ask the kernel to send them pages so they can be cached in the mapper's address space or written back to their segments on disk. This mechanism is needed, for example, to ensure that a group of mappers collectively supporting distributed shared memory can remove writable pages from all machines but one. Other calls in this group relate to locking and unlocking individual pages in memory and getting various sizes and other information about the paging system.

Call	Description
<i>sgRead</i>	Read data from a segment
<i>sgWrite</i>	Write data to a segment
<i>sgStat</i>	Request information about a page cache
<i>sgFlush</i>	Request from a mapper to the kernel asking for dirty pages

Fig. 9-13. Selected calls relating to segments.

The calls in Fig. 9-14 are calls to a mapper, either from the kernel or from an application program asking the mapper to do something for the caller. The first one, *MpCreate* is used when the kernel or a program wants to swap out a segment and needs to allocate disk space for it. The mapper responds by allocating a new segment on disk and returning a capability for it.

Call	Description
<i>MpCreate</i>	Request to create a dummy segment for swapping
<i>MpRelease</i>	Request asking to release a previously created segment
<i>MpPullIn</i>	Request asking for one or more pages
<i>MpPushOut</i>	Request asking mapper to accept one or more pages

Fig. 9-14. Mapper calls.

The *MpRelease* call returns a segment created using *MpCreate*. *MpPullIn* is used by the kernel to acquire data from a newly created or existing segment. The mapper is required to respond to it by sending a message containing the pages needed. By using clever MMU programming, the pages need not be copied physically.

The *MpPushOut* call is for transfers the other way, from kernel to mapper, either in response to a *sgFlush* (or similar) call, or when the kernel wants to

swap out a segment on its own. Although the list of calls described above is not complete, it does give a reasonable picture of how memory management works in Chorus.

9.4. COMMUNICATON IN CHORUS

The basic communication paradigm in Chorus is message passing. During the Version 1 era, when the research was focused on multiprocessors, using shared memory as the communication paradigm was considered, but rejected as not being general enough. In this section we will discuss messages, ports, and the communication operations, concluding with a summary of the kernel calls available for communication.

9.4.1. Messages

Each message contains a header (for the microkernel's internal use only), an optional fixed part and an optional body. The header identifies the source and destination and contains various protection identifiers and flags. The fixed part, if present, is always 64 bytes long and is entirely under user control. The body is variable sized, with a maximum of 64K bytes, and also entirely under user control. From the kernel's point of view, both the fixed part and the body are untyped byte arrays in the sense that the kernel does not care what is in them.

When a message is sent to a thread on a different machine, it is always copied. However, when it is sent to a thread on the same machine, there is a choice between actually copying it and just mapping it into the receiver's address space. In the latter case, if the receiver writes onto a mapped page, a genuine copy is made on the spot (i.e., Mach's copy-on-write mechanism). When a message is not an integral number of pages but the message is mapped, some data just beyond the buffer (or before it) will be lost when the final (or first) page is mapped in.

Another form of message is the **minimessage**, which is only used between kernel processes for short synchronization messages, typically to signal the occurrence of an interrupt. The minimessages are sent to special low-overhead **miniports**.

9.4.2. Ports

Messages are addressed to ports, each of which contains storage for a certain number of messages. If a message is sent to a port that is full, the sender is suspended until sufficient space is available. When a port is created, both a unique identifier and a local identifier are returned to the caller. The former can

be sent to other processes so they can send messages to the port. The latter is used within the process to reference the port directly. Only threads in the process currently holding the port can read from the port (ports can migrate).

When a process is created, it automatically gets a default port that the kernel uses to send it exception messages. It can also create as many additional ports as it needs. These additional ports (but not the default port) can be moved to other processes, even on other machines. When a port is moved, all the messages currently in it can be moved with it. Port movement is useful, for example, when a server on a machine that is going down for maintenance wants to let another server on a different machine take over its work. In this way, services can be maintained in a transparent way, even as server machines go down and come back up again later.

Chorus provides a way to collect several ports together into a **port group**. To do so, a process first creates an empty port group and gets back a capability for it. Using this capability, it can add ports to the group, and later it can use the capability to delete ports from the group. A port may be present in multiple port groups, as illustrated in Fig. 9-15.

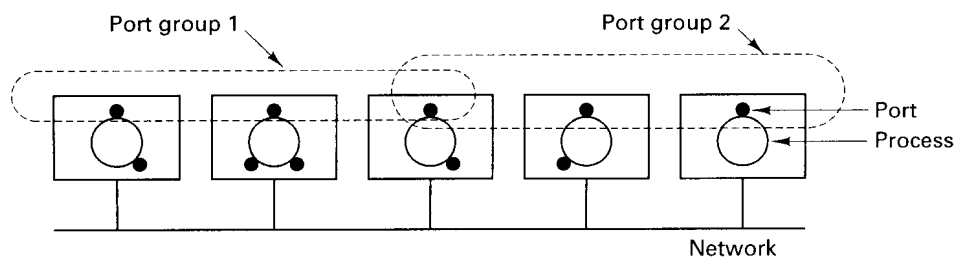


Fig. 9-15. A port may be a member of multiple port groups.

Groups are commonly used to provide reconfigurable services. Initially some set of servers belongs to the group, which provides some service. Clients can send messages to the group without having to know which servers are available to do the work. Later, new servers can join the group and old ones can leave without disrupting the services and without the clients even being aware that the system has been reconfigured.

9.4.3. Communication Operations

Two kinds of communication operations are provided by Chorus: asynchronous send and RPC. Asynchronous send allows a thread simply to send a message to a port. There is no guarantee that the message arrives and no notification if something goes wrong. This is the purest form of datagram and allows users to build arbitrary communication patterns on top of it.

The other communication operation is RPC. When a process performs an RPC operation, it is blocked automatically until either the reply comes in or the RPC timer expires, at which time the sender is unblocked. The message that unblocks the sender is guaranteed to be the response to the request. Any message that does not bear the RPC's transaction identifier will be stored in the port for future consumption but will not awaken the sender.

RPCs use at-most-once semantics, meaning that in the event of an unrecoverable communication or processing failure, the system guarantees that an RPC will return an error code rather than take a chance on having an operation executed more than once.

It is also possible to send a message to a port group. Various options are available, as shown in Fig. 9-16. These options determine how many messages are sent and to which ports.

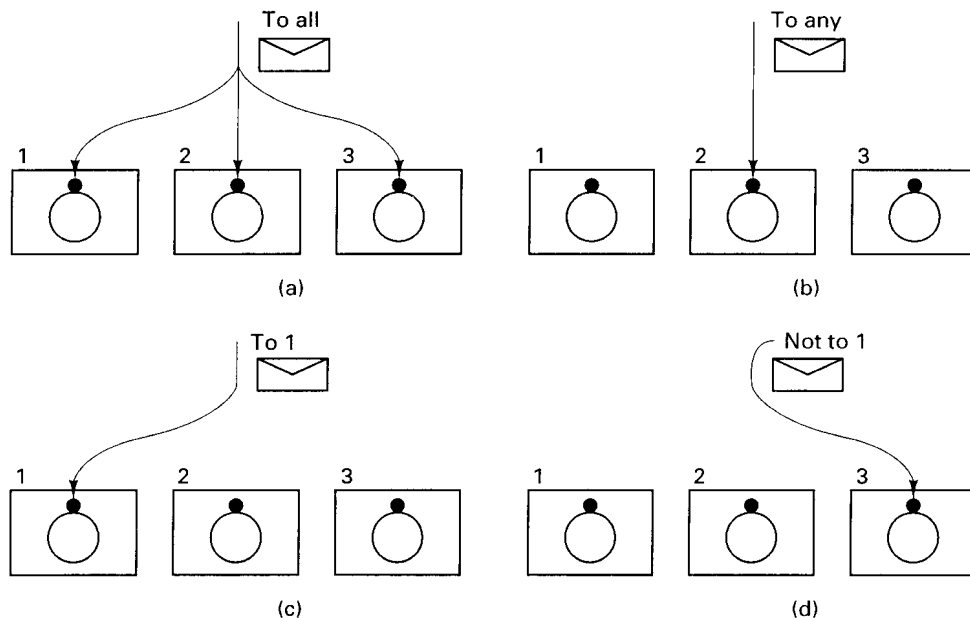


Fig. 9-16. Options for sending to a port group. (a) Send to all members. (b) Send to any member. (c) Send to port at the same site as a given port. (d) Send to a port not at a specific site.

Option (a) in Fig. 9-16 sends the message to all ports in the group. For highly reliable storage, a process might want to have every file server store certain data. Option (b) sends it to just one, but lets the system choose which one. When a process just wants some service, such as the current date, but does not care where it comes from, this option is the best choice, as the system can then select the most efficient way to provide the service.

The other two options also send to just one port, but limit the choice the system may make. In (c), the caller can specify that the port must be on a specific site, for example, to balance the system load. Option (d) says that any port *not* on the specified site may be used. A use for this option might be to force a backup copy of a file onto a different machine than the primary copy.

Sends to port groups use the asynchronous send. Broadcast sends (i.e., to all members) are not flow controlled. If flow control is required, it must be supplied by the user.

To receive a message, a thread makes a kernel call telling which port it wants to receive on. If a message is available, the fixed part of the message is copied to the caller's address space, and the body, if any, is either copied or mapped in, depending on the options. If no message is available, the calling thread is suspended until a message arrives or a user-specified timer expires.

Furthermore, a process can specify that it wants to receive from one of the ports it owns, but it does not care which one. This option can be further refined by disabling some of the ports, in which case only enabled ports are eligible to satisfy the request. Finally, ports can be assigned priorities, which means that if more than one enabled port has a message, the enabled port with the highest priority will be selected. Ports can be enabled and disabled dynamically, and their priorities can be changed at will.

9.4.4. Kernel Calls for Communication

The port management calls are shown in Fig. 9-17. The first four are straightforward, allowing ports to be created, destroyed, enabled, and disabled. The last one specifies a port and a process. After the call completes, the port no longer belongs to its original owner (which need not be the caller) but instead belongs to the target process. It alone can now read messages from the port.

Call	Description
portCreate	Create a port and return its capability
portDelete	Destroy a port
portEnable	Enable a port so its messages count on a receive from all ports
portDisable	Disable a port
portMigrate	Move a port to a different process

Fig. 9-17. Selected port management calls.

Three calls are present for managing port groups. They are listed in Fig. 9-18. The first, *grpAllocate*, creates a new port group and returns a capability for it to the caller. Using this capability, the caller or any other process that subsequently acquires the capability can add or delete ports from the group.

Call	Description
grpAllocate	Create a port group
grpPortInsert	Add a new port to an existing port group
grpPortRemove	Delete a port from a port group

Fig. 9-18. Calls relating to port groups.

Our last group of kernel calls handles the actual sending and receiving of messages. These are listed in Fig. 9-19. *IpcSend* sends a message asynchronously to a specified port or port group. *IpcReceive* blocks until a message arrives from a specified port. This message may have been sent directly to the port, to a port group of which the specified port is a member, or to all enabled ports (assuming that the specified port is enabled). An address into which the fixed part is to be copied must be supplied, but an address for the body is optional, because the size is not always known in advance. If no buffer is provided for the body the *ipcGetData* call can be executed to acquire the body from the kernel (the size is now known since it is returned by the *IpcReceive* call). A reply message can be sent using *ipcReply*. Finally, *ipcCall* performs a remote procedure call.

Call	Description
ipcSend	Send a message asynchronously
ipcReceive	Block until a message arrives
ipcGetData	Get the current message's body
ipcReply	Send a reply to the current message
ipcCall	Perform a remote procedure call

Fig. 9-19. Selected communication calls.

9.5. UNIX EMULATION IN CHORUS

Although being UNIX-like was not even one of the original goals of Chorus at all (it had a totally different interface and was written in interpreted UCSD Pascal), with the advent of Version 3, UNIX compatibility became a major goal. The approach taken is to have the kernel be operating system neutral, but to have a subsystem that allows existing UNIX binary programs to run on it without modification. This subsystem consists partly of new code written by the Chorus implementers, and partly of the System V UNIX code itself, licensed from UNIX System Laboratories. In this section we will describe what Chorus UNIX is like

and how it is implemented. The UNIX subsystem is called **MiX**, which stands for **Modular UNIX**.

9.5.1. Structure of a UNIX Process

A UNIX process runs as a Chorus user process, on top of the UNIX subsystem. As such, it has all the features of a Chorus process, although existing UNIX programs will not use all of them. The standard text, data, and stack segments are implemented by three mapped Chorus segments.

On the whole, the way Chorus works is not hidden from the UNIX runtime system and library (although it is hidden from the *program*). For example, when the program wants to open a file, the library procedure *open* invokes the subsystem, which ultimately gets a file capability. It stores the file capability internal to itself and uses it when the user process calls *read*, *write*, and other procedures that deal with the open file.

Open files are not the only UNIX concepts that are mapped onto Chorus resources. Open directories (including the working directory and root directory), open devices, pipes, and in-use segments are all represented internally as capabilities for the corresponding Chorus resources. Operations on all of these are done by the UNIX subsystem by passing the capability to the appropriate server. To remain binary compatible with UNIX, the user process must never see the capabilities.

9.5.2. Extensions to UNIX

Chorus provides UNIX processes with many extensions to make distributed programming easier. Most of these are just standard Chorus properties that are made visible. For example, UNIX processes can create and destroy new threads using the Chorus threads package. These threads run in quasiparallel (on a multiprocessor, actually in parallel).

Since Chorus threads are managed by the kernel, when one thread blocks, for example on a system call, the kernel is able to schedule and run another thread in the same process. However, it is not usually possible for two threads in the same process to execute most system calls simultaneously because the ancient UNIX code that handles the system calls is not reentrant. In most cases, the second thread is suspended transparently before starting the system call until the first one has completed.

The addition of threads has necessitated rethinking how signals are handled. Instead of all signal handlers being common to the entire process, the synchronous ones (the ones caused by a thread action) are associated with specific threads whereas the asynchronous ones (like the user hitting the DEL key) are process wide. For example, the ALARM system call arranges for the calling

thread to be interrupted after the indicated number of seconds. Other threads are not affected.

Similarly, one thread can arrange to catch floating-point overflows and similar exceptions, while another ignores them, and a third thread does not catch them (meaning that it will be killed if one occurs).

Other signals are, by nature, not specific to one thread. A KILL signal sent by another process, or a SIGINT or SIGQUIT from the keyboard are sent to all threads. Each one can catch or ignore it as it wishes. If no thread catches or ignores a signal, the entire process is killed.

Signals are handled by the process itself. Associated with every UNIX process is a control thread within the UNIX subsystem that spends its day listening to the exception port. When a signal is triggered, this normally dormant thread is awakened. The thread then gets the message sent to the control port, examines its internal tables, and performs the required action, namely interrupting the appropriate thread or threads.

A third area in which UNIX has been extended is in making it distributed. It is possible for a process to make a system call to indicate that new processes are not to be created on the local machine, but on a specific remote machine. When a new process is forked off, it starts on the same machine as the parent process, but when it does an EXEC system call, the new process is started on the remote machine.

User processes using the UNIX subsystem can create ports and port groups and send and receive messages, like any other Chorus processes. They can also create regions and map segments onto them. In general, all the Chorus facilities related to process management, memory management, and interprocess communication are available to UNIX processes as well.

9.5.3. Implementation of UNIX on Chorus

The implementation of the UNIX subsystem is constructed from four principal components: the **process manager**, the **object manager**, the **streams manager**, and the **interprocess communication manager**, as depicted in Fig. 9-20. Each of these has a specific function in the emulation. The process manager catches system calls and does process management. The object manager handles the file system calls and also paging activity. The streams manager takes care of I/O. The interprocess communication manager does System V IPC. The process manager is new code. The others are largely taken from UNIX itself to minimize the designer's work and maximize compatibility. These four managers can each handle multiple sessions, so only one of each is present on a given site, no matter how many users are logged into it.

In the original design, the four processes should have been able to run either in kernel mode or in user mode. However, as more privileged code was added,

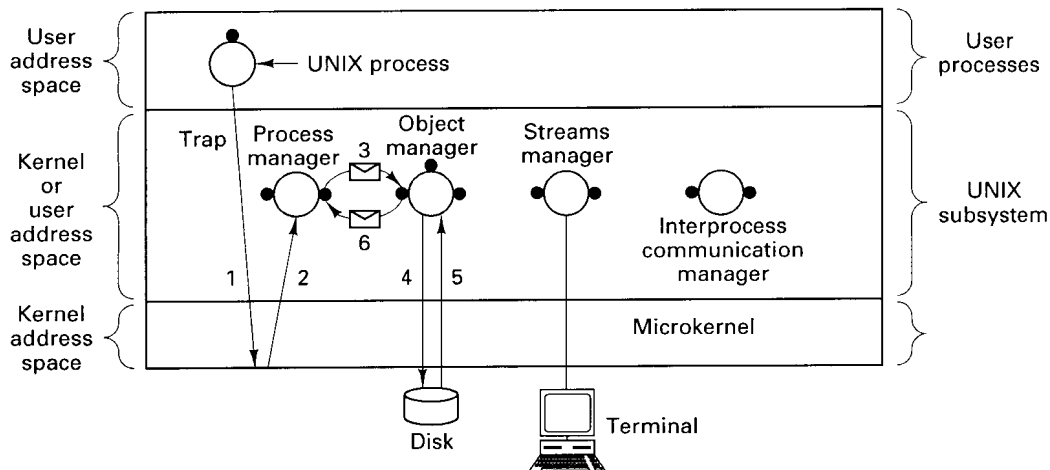


Fig. 9-20. The structure of the Chorus UNIX subsystem. The numbers show the sequence of steps involved in a typical file operation.

this became more difficult to do. In practice now, they normally all run in kernel mode, which also is needed to give acceptable performance.

To see how the pieces relate, we will examine how system calls are processed. At system initialization time, the process manager tells the kernel that it wants to handle the trap numbers standard AT&T UNIX uses for making system calls (to achieve binary compatibility). When a UNIX process later issues a system call by trapping to the kernel, as indicated by (1) in Fig. 9-21, a thread in the process manager gets control. This thread acts as though it is the same as the calling thread, analogous to the way system calls are made in traditional UNIX systems. Depending on the system call, the process manager may perform the requested system call itself, or as shown in Fig. 9-20, send a message to the object manager asking it to do the work. For I/O calls, the streams manager is invoked. For IPC calls, the communication manager is used.

In this example, the object manager does a disk operation and then sends a reply back to the process manager, which sets up the proper return value and restarts the blocked UNIX process.

The Process Manager

The process manager is the central player in the emulation. It catches all system calls and decides what to do with them. It also handles process management (including creating and terminating processes), signals (both generating them and receiving them), and naming. When a system call that it does not

handle comes in, the process manager does an RPC to either the object manager or streams manager. It can also make kernel calls to do its job; for example, when forking off a new process, the kernel does most of the work.

If the new process is to be created on a remote machine, a more complicated mechanism is required, as shown in Fig. 9-21. Here the process manager catches the system call, but instead of asking the local kernel to create a new Chorus process, it does an RPC to the process manager on the target machine, step (3) in Fig. 9-21. The remote process manager then asks its kernel to create the process, as shown in step 4. Each process manager has a dedicated port to which RPCs from other process managers are directed.

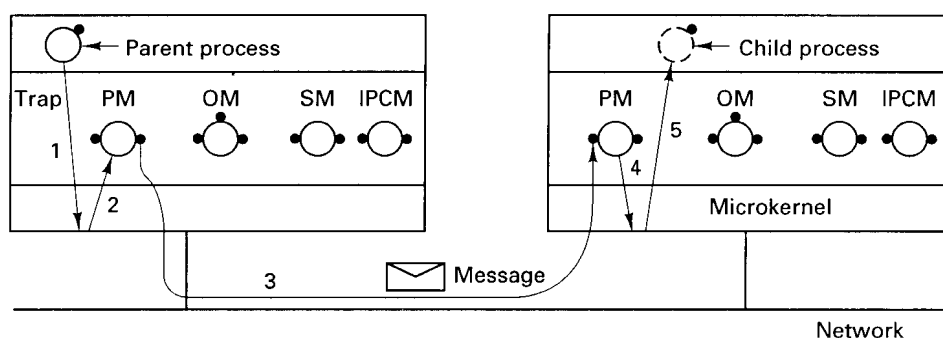


Fig. 9-21. Creating a process on a remote machine. The reply message is not shown.

The process manager has multiple threads. For example, when a user thread sets an alarm, a process manager thread goes to sleep until the timer goes off. Then it sends a message to the exception port belonging to the thread's process.

The process manager also manages the unique identifiers. It maintains tables that map the UIs onto the corresponding resources.

The Object Manager

The object manager handles files, swap space, and other forms of tangible information. It may also contain the disk driver. In addition to its exception port, the object manager has a port for receiving paging requests and a port for receiving requests from local or remote process managers. When a request comes in, a thread is dispatched to handle it. Several object manager threads may be active at once.

The object manager acts as a mapper for the files it controls. It accepts page fault requests on a dedicated port, does the necessary disk I/O, and sends appropriate replies.

The object manager works in terms of segments named by capabilities, in

other words, in the Chorus way. When a UNIX process references a file descriptor, its runtime system invokes the process manager, which uses the file descriptor as an index into a table to locate the capability corresponding to the file's segment.

Logically, when a process reads from a file, that request should be caught by the process manager and then forwarded to the object manager using the normal RPC mechanism. However, since reads and writes are so important, an optimized strategy is used to improve their performance. The process manager maintains a table with the segment capabilities for all the open files. It makes an *sgRead* call to the kernel to get the required data. If the data are available, the kernel copies them directly to the user's buffer. If they are not, the kernel makes an *MpPullIn* upcall to the appropriate mapper (usually, the object manager). The mapper issues one or more disk reads, as needed. When the pages are available, the mapper gives them to the kernel, which copies the required data to the user's buffer and completes the system call.

The Streams Manager

The streams manager handles all the System V streams, including the keyboard, display, mouse, and tape devices. During system initialization, the streams manager sends a message to the object manager, announcing its port and telling which devices it is prepared to handle. Subsequent requests for I/O on these devices can then be sent to the streams manager.

The streams manager also handles Berkeley sockets and networking. In this way, a process on Chorus can communicate using TCP/IP as well as other network protocols. Pipes and named pipes are also handled here.

The Interprocess Communication Manager

This process handles those system calls relating to System V messages (not Chorus messages), System V semaphores (not Chorus semaphores), and System V shared memory (not Chorus shared memory). The system calls are unpleasant and the code is taken mostly from System V itself. The less said about them, the better.

Configurability

The division of labor within the UNIX subsystem makes it relatively straightforward to configure a collection of machines in different ways so that each one only has to run the software it needs. The nodes of a multiprocessor can also be configured differently. All machines need the process manager, but the other managers are optional. Which managers are needed depends on the application.

Let us now briefly discuss several different configurations that can be built using Chorus. Figure 9-22(a) shows the full configuration, which might be used on a workstation (with a hard disk) connected to a network. All four of the UNIX subsystem processes are required and present.

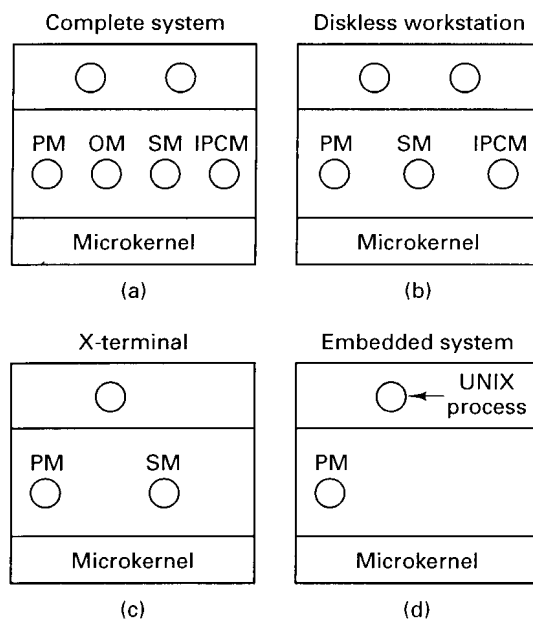


Fig. 9-22. Different applications may best be handled using different configurations.

The object manager is needed only on machines containing a disk, so on a diskless workstation the configuration of Fig. 9-22(b) would be most appropriate. When a process reads or writes a file on this machine, the process manager forwards the request to the object manager on the user's file server. In principle, either the user's machine or the file server can do caching, but not both, except for segments that have been opened or mapped in read-only mode.

For dedicated applications, such as an X-terminal, it is known in advance which system call the program may do and which it may not do. If no local file system and no System V IPC are needed, the object manager and interprocess communication manager can be omitted, as in Fig. 9-23(c).

Finally, for a disconnected embedded system, such as the controller for a car, television set, or talking teddy bear, only the process manager is needed. The others can be left out to reduce the amount of ROM needed. It is even possible for a dedicated application program to run directly on top of the microkernel.

The configuration can be done dynamically. When the system is booted it

can inspect its environment and determine which of the managers are needed. If the machine has a disk, the object manager is loaded; otherwise, it is not, and so on. Alternatively, configuration files can be used. Also, each manager can be created with one thread. As requests come in, additional threads can be created on-the-fly. Table space, for example, the process table, is allocated dynamically from a pool, so it is not necessary to have a different kernel binary for small systems, with only a few processes, and large ones, with thousands of processes.

Real-Time Applications

Chorus has been designed to handle real-time applications, with or without UNIX. The Chorus scheduler, in particular, reflects this goal. Priorities range from 1 to 255. The lower the number, the higher the priority, as in UNIX.

Ordinary UNIX application processes run at priorities 128 to 255, as shown in Fig. 9-23. At these priority levels, when a process uses up its CPU quantum, it is put on the end of the queue for its priority level. The processes that make up the UNIX subsystem run at priorities 64 through 68. Thus a large number of priorities are available both higher and lower than the ones used by the UNIX subsystem for real-time processes.

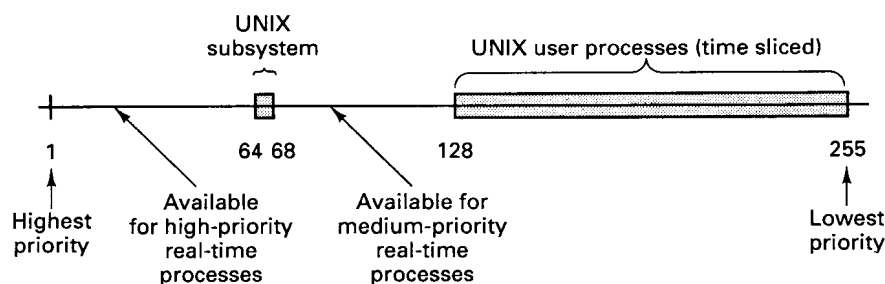


Fig. 9-23. Priorities and real-time processes.

Another facility that is important for real-time work is the ability to reduce the amount of time the CPU is disabled after an interrupt. Normally, when an interrupt occurs, an object manager or streams manager thread does the required processing immediately. However, an option is available to have the interrupt thread arrange for another, lower-priority thread to do the real work, so the interrupt can terminate almost immediately. Doing this reduces the amount of dead time after an interrupt, but it also increases the overhead of interrupt processing, so this feature must be used with care.

These facilities merge well with UNIX, so it is possible to debug a program under UNIX in the usual way, but have it run as a real-time process when it goes into production just by configuring the system differently.

9.6. COOL: AN OBJECT-ORIENTED SUBSYSTEM

The UNIX subsystem is really nothing more than a collection of Chorus processes that are marked as a subsystem. Consequently, it is possible to have other subsystems running at the same time. A second subsystem that has been developed for Chorus is **COOL (Chorus Object-Oriented Layer)**. It was designed for research on object-oriented programming and to bridge the gap between coarse-grained system objects, such as files, and fine-grained language objects, such as structures (records). We will describe COOL in this section. For more information, see (Lea et al., 1991, 1993).

Work on the first version, COOL-1, began in 1988. The goal was to provide system-level support for fine-grained object-oriented languages and applications, and do so in such a way that new COOL programs and old UNIX programs could run side by side on the same machine without interfering.

Each Object in COOL-1 consisted of two Chorus segments, one for the data and one for the code. Programs did not access the data segments directly. Instead they invoked procedures, called **methods**, located in the objects' code segments. In this way, the objects' internal representation was hidden from the user, allowing object writers the freedom to use whatever representation seemed best (for example, an array or a linked list). This representation could even be changed later, without programs using the objects even knowing. Multiple objects of the same class shared the same code segment, to save memory.

To make a long story short, the resulting system was disappointing in terms of performance, resource usage, and so on, and the gap between coarse-grained system objects and fine-grained language objects was not bridged. In 1990, the designers started over and designed COOL-2. This system was running a year later. Below we will describe its architecture and implementation.

9.6.1. The COOL Architecture

Conceptually COOL provides a **COOL base layer** that spans machine boundaries. This layer provides a form of address space that is visible to all COOL processes without regard to where they are running, much like a distributed file system provides a global file space. On top of this layer is the **generic runtime system**, which is also system wide. Above that are the language runtime systems and then the user programs, as illustrated in Fig. 9-24.

9.6.2. The COOL Base Layer

The COOL base provides a set of services for COOL user processes, specifically for the COOL generic library that is linked with each COOL process. The most important service is a memory abstraction, roughly analogous to

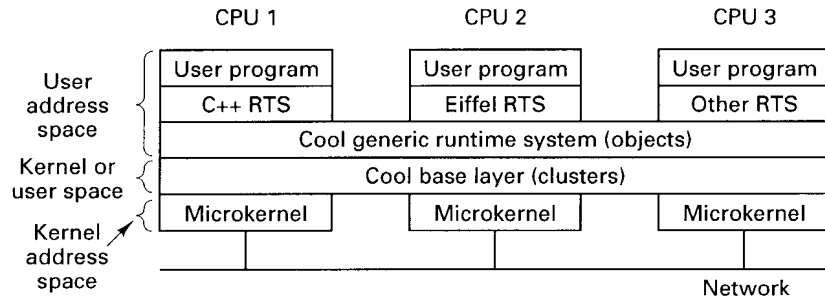


Fig. 9-24. The COOL architecture.

distributed shared memory, but more tuned to object-oriented programming. This abstraction is based on the **cluster**, which is a set of Chorus regions backed by segments. Each cluster normally holds a group of related objects, for example, objects belonging to the same class. It is up to the upper layers of software to determine which objects go in which cluster.

A cluster can be mapped into the address spaces of multiple processes, possibly on different machines. A cluster always begins on a page boundary, and occupies the same addresses in all processes that are currently using it. The regions in a cluster need not be contiguous in virtual address space, so a cluster may, for example, occupy addresses 1024-2047, 4096-8191, and 14336-16535 (assuming a 1K page size).

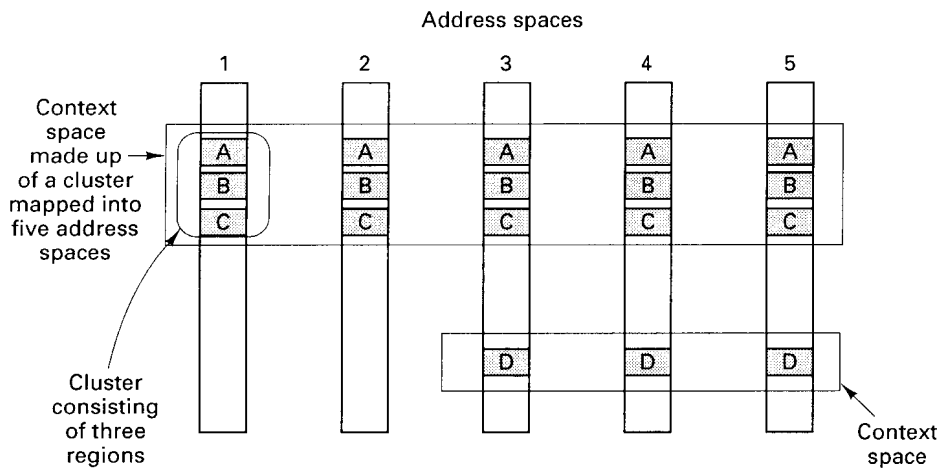


Fig. 9-25. Context spaces and clusters.

A second concept supported by the COOL base is the **context space**, which is a collection of address spaces, again, possibly on multiple machines. Figure

9-25 shows a system with five address spaces (on up to five machines) and two context spaces. The first context space spans all the machines and contains a cluster with three regions. The threads living in these address spaces can all access the objects in this cluster as though they were in a shared memory. The second context space spans only three machines and has a cluster with one region. Threads in address spaces 1 and 2 cannot access the objects in this cluster. The corresponding clusters in different address spaces must be mapped in at the same virtual addresses. Internal pointers are relocated at the time the mapping occurs.

Clusters are not replicated. This means that although a cluster may appear in multiple address spaces at the same time, there is only one physical copy of each cluster. When a user thread tries to invoke a method on an object that is in its address space but physically not present on its machine, a trap occurs to the COOL base. The base then either forwards the request to the machine holding the cluster for remote invocation or arranges for the cluster to migrate to its machine for local invocation.

9.6.3. The COOL Generic Runtime System

The COOL generic runtime uses clusters and context spaces to manage objects. Objects are persistent and exist on disk when no process has them mapped in. Operations are provided to the language runtime system to create and delete objects, map them into and out of address spaces, and invoke their methods. When an object invokes a method of another object that is located in its own cluster, the invocation is done by a local procedure call. When the invoked object is in a different cluster, the generic runtime system is used to ask the COOL base to arrange for the remote invocation. Because the overhead of intracluster invocations is so much smaller than that of intercluster invocations, putting objects that invoke each other a lot together in one cluster greatly reduces system overhead. In COOL-1, all invocations were effectively intercluster operations, which proved to be expensive.

The generic runtime system has a standard interface to the language-dependent runtime systems that use it. The interface is based on **up calls**, in which the generic runtime system calls the language runtime system to find out properties of objects, for example, to find internal pointers for relocation purposes when an object is mapped in.

9.6.4. The Language Runtime System

Normally, when programmers define objects, they define them in a special interface definition language. These definitions then are compiled into objects that can be called at run time to invoke the objects. These interface objects then

decide whether invocations on remote objects will be done remotely, or the object will be migrated locally. Methods in the interface objects allow the program to control the policy used.

9.6.5. Implementation of COOL

The COOL subsystem runs on top of the Chorus microkernel, just like the UNIX one. It consists of a process that implements the COOL base and a COOL generic runtime system that is linked with every COOL program (along with the language library), as shown in Fig. 9-26. This design allows COOL processes to make calls to the COOL subsystem, while at the same time UNIX processes make calls to the UNIX subsystem, without the two interfering. This modularity is a direct result of the microkernel design, and holds for Amoeba and Mach as well, of course.

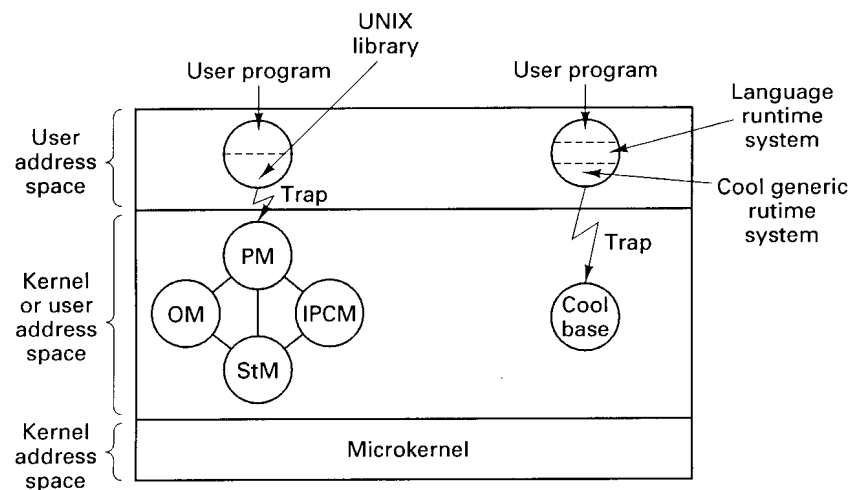


Fig. 9-26. Implementation of COOL.

9.7. COMPARISON OF AMOEBA, MACH, AND CHORUS

In this and the preceding two chapters we have looked at three microkernel-based distributed operating systems, Amoeba, Mach, and Chorus, in considerable detail. While they have some points in common, Amoeba and Mach differ in many of the technical details. Chorus has borrowed many ideas from both Amoeba and Mach, and thus often takes an intermediate position between the

two. In this section we will look at all three systems side-by-side to illustrate the various choices that designers can make.

9.7.1. Philosophy

Amoeba, Mach and Chorus have different histories and different philosophies. Amoeba was designed from scratch as a distributed system for use on a collection of CPUs connected by a LAN. Later, multiprocessors and WANs were added. Mach (actually RIG) started out as an operating system for a single processor, with multiprocessors and LANs being added afterward. Chorus began as a distributed operating systems research project quite far from the UNIX world, and went through three major revisions, getting closer and closer to UNIX each time. The consequences of these differing backgrounds are still visible in the systems.

Amoeba is based on the processor pool model. A user does not log into a particular machine, but into the system as a whole. The operating system decides where to run each command, based on the current load. It is normal for requests to use multiple processors; rarely will two consecutive commands run on the same processor. There is no concept of a home machine.

In contrast, Mach and Chorus (UNIX) were designed with the idea that a user definitely logs into a specific machine and runs all his programs there by default. There is no attempt to spread each user's work over as many machines as possible (although on a multiprocessor, the work will be spread automatically over all the multiprocessor's CPUs). While it is possible to run remotely, the philosophical difference is that each user has a home machine (e.g., a workstation) where most of his work is carried out. However, this distinction was later blurred when Mach was ported to the Intel Paragon, a machine consisting of a pool of processors.

Another philosophical difference relates to what a "microkernel" is. The Amoeba view follows the famous dictum expounded by the French aviator and writer Antoine de St. Exupéry: Perfection is not achieved when there is nothing left to add, but when there is nothing left to take away. Whenever a proposal was made to add a new feature to the kernel, the deciding question was: Can we live without it? This philosophy led to a minimal kernel, with most of the code in user-level servers.

The Mach designers, in contrast, wanted to provide enough functionality in the kernel to handle the widest possible range of applications. In many areas, Amoeba contains one way to do something and Mach contains two or three, each more convenient or efficient under different circumstances. Consequently, the Mach kernel is much larger and has five times as many system calls (including calls to kernel threads) as Amoeba. Chorus occupies an intermediate

position between Amoeba and Mach, but it still has more system calls than 4.2 BSD UNIX, which is hardly a microkernel. A comparison is given in Fig. 9-27.

System	Kernel calls
Amoeba	30
Version 7 UNIX	45
4.2 BSD	84
Chorus	112
Mach	153
SunOS	165

Fig. 9-27. Number of system calls and calls to kernel threads in selected systems.

Another philosophical difference between Amoeba and Mach is that Amoeba has been optimized for the remote case (communication over the network) and Mach for the local case (communication via memory). Amoeba has extremely fast RPC over the network, while Mach's copy-on-write mechanism provides high-speed communication on a single node, for example. Chorus has primarily emphasized single CPU and multiprocessor systems, but since the communication management is done in the kernel (as in Amoeba) and not in a user process (as in Mach), it potentially can have good RPC performance, too.

9.7.2. Objects

Objects are the central concept in Amoeba. A few are built in, like threads and processes, but most are user defined (e.g., files) and can have arbitrary operations on them. About a dozen generic operations (e.g., get status) are defined on nearly all objects, with various object-specific operations defined on each one as well.

In contrast, the only objects supported directly by Mach are threads, processes, ports, and memory objects, each with a fixed set of operations. Higher level software can use these concepts to build other objects, but they are qualitatively different than the built-in objects like memory objects.

In all three systems, objects are named, addressed, and protected by capabilities. In Amoeba, capabilities are managed in user space and protected by one-way functions. Capabilities for system-defined objects (e.g., processes) and for user-defined objects (e.g., directories) are treated in a uniform way and appear in user-level directories for naming and addressing all objects. Amoeba capabilities are in principle worldwide, that is, a directory can hold capabilities for files and other objects that are located anywhere. Objects are located by broadcasting, with the results cached for future use.

Mach also has capabilities, but only for ports. These are managed by the kernel in capability lists, one per process. Unlike Amoeba, there are no capabilities for processes or other system or user-defined objects, and they are not generally used directly by application programs. Port capabilities are passed between processes in a controlled way, so Mach can find them by looking them up in kernel tables.

Chorus supports about the same built-in objects as Mach, but also uses Amoeba's capability system for allowing subsystems to define new protected objects. Unlike Amoeba, Chorus capabilities do not have an explicit (cryptographically protected) field giving the allowed rights. Like Amoeba's capabilities and unlike Mach's, Chorus' capabilities can be passed from one process to another simply by including them in a message or writing them to a shared file.

9.7.3. Processes

All three systems support processes with multiple threads per process. Again in all three cases, the threads are managed and scheduled by the kernel, although a user-level threads packages can be built on top of them. Amoeba does not provide user control over thread scheduling, whereas Mach and Chorus allows processes to set the priorities and policies of their threads in software. Mach provides more elaborate multiprocessor support than the other two.

In Amoeba and Chorus, synchronization between threads is done by mutexes and semaphores. In Mach it is done by mutexes and condition variables. Amoeba and Mach support some form of global variables. Chorus uses software registers to define each thread's private context.

Amoeba, Mach, and Chorus all work on multiprocessors, but they differ on how they deal with threads on these machines. In Amoeba, all the threads of a process run on the same CPU, in pseudoparallel, timeshared by the kernel. In Mach processes have fine-grained control over which threads run on which CPUs (using the processor set concept). Consequently, the threads of the same process can run in parallel on different CPUs.

In Amoeba, a similar effect can be achieved by having several single-threaded processes run on different CPUs and share a common address space. Nevertheless, it is clear that the Mach designers have devoted more attention to multiprocessors than have the Amoeba designers.

Chorus supports having multiple threads within a single process running on different CPUs at the same time, but this is handled entirely by the operating system. There are no user-visible primitives for managing thread-to-processor allocation, but there will be in the future.

However, the Amoeba designers have put more work into supporting load balancing and heterogeneity. When the Amoeba shell starts a process, it asks the run server to find it the CPU with the lightest workload. Unless the user has

specified a specific architecture, the process may be started on any architecture for which a binary is available, with the user not even being aware which kind has been selected. This scheme is designed to spread the workload over as many machines as possible all the time.

In Mach, processes are normally started on the user's home machine. Only when explicitly requested to do so by the user are processes run remotely on idle workstations, and even then they have to be evicted quickly if the workstation's owner touches the keyboard. This difference relates to the fundamental difference between the processor pool model and the workstation model.

Chorus allows a process to be started on any machine. The UNIX emulation provides a way to set the default site.

9.7.4. Memory Model

Amoeba's memory model is based on variable-length segments. A virtual address space consists of some number of segments mapped in at specific addresses. Segments can be mapped in and out at will. Each segment is controlled by a capability. A remote process that has the capabilities for another process' segments (e.g., a debugger) can read and write them from any other Amoeba machine. Amoeba does not support paging. When a process is running, all of its segments are in memory. The ideas behind this decision are simplicity of design and high performance, coupled with the fact that extremely large memories are becoming common on even the smallest machines.

Mach's memory model is based on memory objects and is implemented in terms of fixed-size pages. Memory objects can be mapped and unmapped at will. A memory object need not be entirely in main memory to be used. When an absent page is referenced, a page fault occurs and a message is sent to an external memory manager to find the page and map it in. Together with the default memory manager, this mechanism supports paged virtual memory.

Pages can be shared between multiple processes in various ways. One common configuration is the copy-on-write sharing used to attach a child process to its parent. Although this mechanism is a highly efficient way of sharing on a single node, it loses its advantages in a distributed system because physical transport is always required (assuming that the receiver needs to read the data). In such an environment, the extra code and complexity are wasted. This is a clear example of where Mach has been optimized for single-CPU and multiprocessor systems, rather than for distributed systems.

Chorus' memory management model is taken largely from Mach. It too has memory objects (segments) that can be mapped in. These are demand paged under the control of an external pager (mapper), as in Mach.

Amoeba, Mach, and Chorus all support distributed shared memory, but they do it in different ways. Amoeba supports shared objects that are replicated on

all machines using them. Objects can be of any size and can support any operations. Reads are done locally, and writes are done using the Amoeba reliable broadcast protocol.

Mach and Chorus, in contrast, support a page-based distributed shared memory. When a thread references a page that is not present on its machine, the page is fetched from its current machine and brought in. If two machines heavily access the same writable page, thrashing can occur. The trade-off here is the more expensive update on Amoeba (due to the replication of writable objects), versus the potential for thrashing on Mach and Chorus (only one copy of writable pages).

9.7.5. Communication

Amoeba supports both RPC and group communication as fundamental primitives. RPCs are addressed to put-ports, which are service addresses. They are protected cryptographically using one-way functions. The sending and receiving machines can be anywhere. The RPC interface is very simple: only three system calls, none with any options.

Group communication provides reliable broadcasting as a user primitive. Messages can be sent to any group with a guarantee of reliable delivery. In addition, all group members see all messages arrive in exactly the same order.

Low-level communication uses the FLIP protocol, which provides process addressing (as opposed to machine addressing). This feature allows process migration and (inter)network reconfiguration automatically, without the software even being aware of it. It also supports other facilities useful in distributed systems.

In contrast, Mach's communication is from process to port, rather than from process to process. Furthermore, the sender and port must be on the same node. Using a network message server or the NORMA code as a proxy, communication can be extended over a network, but this indirection extracts a performance penalty. Mach does not support group communication or reliable broadcasting as basic kernel primitives.

Communication is done using the *mach_msg* system call, which has seven parameters, ten options, and 35 potential error messages. It supports both synchronous and asynchronous message passing. This approach is the antithesis of the Amoeba strategy of "Keep it simple and make it fast." The idea here is to provide the maximum flexibility and the widest range of support for present and future applications.

Mach messages can be either simple or complex. Simple messages are just bits and are not processed in any special way by the kernel. Complex messages may contain capabilities. They may also pass out-of-line data using copy-on-write, something Amoeba does not have. On the other hand, this facility is of

little value in a distributed system because the out-of-line data must be fetched by the network message server, combined with the message header and in-line data, and sent over the network. This optimization is for the local case and wins nothing when the sender and receiver are on different machines.

On the network, Mach uses conventional protocols such as TCP/IP. These have the advantage of being stable and widely available. FLIP, in contrast, is new, but is faster for typical RPC usage and has been specifically designed for the needs of distributed computing.

Communication in Chorus is philosophically similar to Mach, but simpler. Messages are directed to ports and can either be sent asynchronously or by using RPC. All Chorus communication is handled by the kernel, as in Amoeba. There is nothing like the network message server in Chorus. Like Amoeba and unlike Mach, a Chorus message has a fixed header containing source and destination information, and an untyped body that is just an array of bytes as far as the system is concerned. As in Amoeba, capabilities in messages are not treated in any special way.

Chorus supports broadcasting at the kernel level but in a way more like Mach (which does not support it) than like Amoeba (which does). Ports can be grouped together in port groups, and messages sent to all the members of a port group (or to just one). There is no guarantee that all processes see all messages in the same order, as in Amoeba.

9.7.6. Servers

Amoeba has a variety of servers for specific functions, including file management, directory management, object replication, and load balancing. All are based on objects and capabilities. Amoeba supports replicated objects via directories that contain capability sets. UNIX emulation is provided at the source code level, is based on POSIX, but is not 100 percent complete. The emulation is done by mapping UNIX concepts onto Amoeba ones and calling on the native Amoeba servers to do the work. For example, when a file is opened, the capability for the file is acquired and stored in the file descriptor table.

Mach has a single server that runs BSD UNIX as an application program. It provides 100 percent binary-compatible emulation, a great boon for running existing software for which the source code is not available. General object replication is not supported. Other servers also exist.

Chorus provides full binary compatibility with System V UNIX. The emulation is done by a collection of processes (like Amoeba) rather than by running UNIX as an application program (like Mach). However, some of these processes contain actual UNIX code, like Mach and unlike Amoeba. Like Amoeba, the native servers were designed from scratch with distributed computing in mind, so the emulation translates the UNIX constructs onto the native ones. Just as in

Amoeba, for example, when a file is opened, a capability for it is fetched and stored in the file descriptor table.

A brief summary of some of the points discussed above is given in Fig. 9-28.

Item	Amoeba	Mach	Chorus
Designed for:	Distributed system	1 CPU, multiprocessor	1 CPU, multiprocessor
Execution model	Pool processor	Workstation	Workstation
Microkernel?	Yes	Yes	Yes
Number of kernel calls	30	153	112
Automatic load balancing?	Yes	No	No
Capabilities	General	Only ports	General
Capabilities in:	User space	Kernel	User space
Threads managed by:	Kernel	Kernel	Kernel
Transparent heterogeneity?	Yes	No	No
User-settable priorities?	No	Yes	Yes
Multiprocessor support	Minimal	Extensive	Moderate
Mapped object	Segment	Memory object	Segment
Demand paging?	No	Yes	Yes
Copy on write?	No	Yes	Yes
External pagers?	No	Yes	Yes
Distributed shared memory	Object based	Page based	Page based
RPC?	Yes	Yes	Yes
Group communication	Reliable, ordered	None	Unreliable
Asynchronous communication?	No	Yes	Yes
Intermachine messages	Kernel	User space/ kernel	Kernel
Messages address to:	Process	Port	Port
UNIX emulation	Source	Binary	Binary
UNIX compatibility	POSIX (partial)	BSD	System V
Single-server UNIX?	No	Yes	No
Multiserver UNIX?	Yes	No	Yes
Optimized for:	Remote case	Local case	Local case
Automatic file replication?	Yes	No	No

Fig. 9-28. A comparison of Amoeba, Mach, and Chorus.

9.8. SUMMARY

Like Amoeba and Mach, Chorus is a microkernel-based operating system for use in distributed systems. It provides binary compatibility with System V UNIX, support for real-time applications, and object-oriented programming.

Chorus consists of three conceptual layers: the kernel layer, the subsystems, and the user processes. The kernel layer contains the microkernel proper, as well as some kernel processes that run in kernel mode and share the microkernel's address space. The middle layer contains the subsystems, which are used to build provide operating system support to user programs, which reside in the top layer.

The microkernel provides six key abstractions: processes, threads, regions, messages, ports, port groups, and unique identifiers. Processes provide a way to collect and manage resources. Threads are the active entities in the system, and are scheduled by the kernel using a priority-based scheduler. Regions are areas of virtual address space that can have segments mapped into them. Ports are buffers used to hold incoming messages not yet read. Unique identifiers are binary names used to identifying resources.

The microkernel and subsystems together provide three additional constructs: capabilities, protection identifiers, and segments. The first two are used to name and protect subsystem resources. The third is the basis of memory allocation, both within a running process and on disk.

Two subsystems were described in this chapter. The UNIX subsystem consists of the process, object, streams, and interprocess communication managers, which work together to provide binary compatible UNIX emulation. The COOL subsystem provides support for object-oriented programming.

PROBLEMS

1. Capabilities in Chorus use epoch numbers in their UIs. Why?
2. What is the difference between a region and a segment?
3. The Chorus supervisor is machine dependent, whereas the real-time executive is machine independent. Explain.
4. Why does Chorus need system processes in addition to user processes and kernel processes?
5. What is the difference between a thread being `SUSPENDED` and it being `STOPPED`? After all, in both cases it cannot run.
6. Briefly describe how exceptions and interrupts are handled in Chorus, and tell why they are handled differently.
7. Chorus supports both semaphores and mutexes. Is this strictly necessary? Would it not be sufficient to support only semaphores?

8. What is the function of a mapper?
9. Briefly describe what *MpPullIn* and *MpPushOut* are used for.
10. Chorus supports both RPC and an asynchronous send. What is the essential difference between these two?
11. Give a possible use of port migration in Chorus.
12. It is possible to send a message to a port group in Chorus. Does the message go to all the ports in the group, or to a randomly selected port?
13. Chorus has explicit calls to create and delete ports, but only a call for creating port groups (*grpAllocate*). Make an educated guess as to why there is no *grpDelete*.
14. Why were miniports introduced? Do they do anything that regular ports do not do?
15. Why does Chorus support both preemptive and nonpreemptive scheduling?
16. Name one way in which Chorus is like Amoeba. Name one way in which it is like Mach.
17. How does Chorus' use of port groups differ from group communication in Amoeba?
18. Why did Chorus extend the semantics of UNIX signals?

10

Case Study 4: DCE

In the preceding three chapters we have looked at microkernel-based distributed systems in some detail. In this chapter we will examine a completely different approach, the Open System Foundation's **Distributed Computing Environment**, or **DCE** for short. Unlike the microkernel-based approaches, which are revolutionary in nature—throwing out current operating systems and starting all over—DCE takes an evolutionary approach, building a distributed computing environment on top of existing operating systems. In the next section we will introduce the ideas behind DCE, and in those following it, we will look at each of the principal components of DCE in some detail.

10.1. INTRODUCTION TO DCE

In this section we will give an overview of the history, goals, models, and components of DCE, as well as an introduction to the cell concept, which plays an important role in DCE.

10.1.1. History of DCE

OSF was set up by a group of major computer vendors, including IBM, DEC, and Hewlett Packard, as a response to AT&T and Sun Microsystems

signing an agreement to further develop and market the UNIX operating system. The other companies were afraid that this arrangement would give Sun a competitive advantage over them. The initial goal of OSF was to develop and market a new version of UNIX, over which they, and not AT&T/Sun, had control. This goal was accomplished with the release of OSF/1.

From the beginning it was apparent that many the OSF consortium's customers wanted to build distributed applications on top of OSF/1 and other UNIX systems. OSF responded to this need by issuing a "Request for Technology" in which they asked companies to supply tools and other software needed to put together a distributed system. Many companies made bids, which were carefully evaluated. OSF then selected a number of these offerings, and developed them further to produce a single integrated package—DCE—that could run on OSF/1 and also on other systems. DCE is now one of OSF's major products. A complementary product, **DME (Distributed Management Environment)**, for managing distributed systems was planned but never made it.

10.1.2. Goals of DCE

The primary goal of DCE is to provide a coherent, seamless environment that can serve as a platform for running distributed applications. Unlike Amoeba, Mach, and Chorus, this environment is built on top of existing operating systems, initially UNIX, but later it was ported to VMS, WINDOWS, and OS/2. The idea is that the customer can take a collection of existing machines, add the DCE software, and then be able to run distributed applications, all without disturbing existing (nondistributed) applications. Although most of the DCE package runs in user space, in some configurations a piece (part of the distributed file system) must be added to the kernel. OSF itself only sells source code, which vendors integrate into their systems. For simplicity, in this chapter we will concentrate primarily on DEC on top of UNIX.

The environment offered by DCE consists of a large number of tools and services, plus an infrastructure for them to operate in. The tools and services have been chosen so that they work together in an integrated way and make it easier to develop distributed applications. For example, DCE provides tools that make it easier to write applications that have high availability. As another example, DCE provides a mechanism for synchronizing clocks on different machines, to yield a global notion of time.

DCE runs on many different kinds of computers, operating systems, and networks. Consequently, application developers can easily produce portable software that runs on a variety of platforms, amortizing development costs and increasing the potential market size.

The distributed system on which a DCE application runs can be a heterogeneous system, consisting of computers from multiple vendors, each of which

has its own local operating system. The layer of DCE software on top of the operating system hides the differences, automatically doing conversions between data types when necessary. All of this is transparent to the application programmer.

As a consequence of all of the above, DCE makes it easier to write applications in which multiple users at multiple sites work together, collaborating on some project by sharing hardware and software resources. Security is an important part of any such arrangement, so DCE provides extensive tools for authentication and protection.

Finally, DCE has been designed to interwork with existing standards in a number of areas. For example, a group of DCE machines can communicate with each other and with the outside world using either TCP/IP or the OSI protocols, and resources can be located either using the DNS or X.500 naming systems. The POSIX standards are also used.

10.1.3. DCE Components

The programming model underlying all of DCE is the client/server model. User processes act as clients to access remote services provided by server processes. Some of these services are part of DCE itself, but others belong to the applications and are written by the applications programmers. In this section we will give a quick introduction to those distributed services that are provided by the DCE package itself, mainly the time, directory, security, and file system services.

In most DCE applications, client programs are more-or-less normal C programs that have been linked with a special library. The client binary programs then contain a small number of library procedures that provide the interfaces to the services, hiding the details from the programmers. Servers, in contrast, are normally large daemon programs. They run all the time, waiting for requests for work to arrive. When requests come in, they are processed and replies are sent back.

In addition to offering distributed services, DCE also furnishes two facilities for distributed programming that are not organized as services: threads and RPC (Remote Procedure Call). The threads facility allows multiple threads of control to exist in the same process at the same time. Some versions of UNIX provide threads themselves, but using DCE provides a standard thread interface across systems. Where possible, DCE can use the native threads to implement DCE threads, but where there are no native threads, DCE provides a threads package from scratch.

The other DCE facility is RPC, which is the basis for all communication in DCE. To access a service, the client process does an RPC to a remote server process. The server carries out the request and (optionally) sends back a reply.

DCE handles the complete mechanism, including locating the server, binding to it, and performing the call.

Figure 10-1 gives an approximate idea of how the various parts of DCE fit together. At the lowest level is the computer hardware, on top of which the native operating system (with DCE additions) runs. To support a full-blown DCE server, the operating system must either be UNIX or be another system with essentially the functionality of UNIX, including multiprogramming, local inter-process communication, memory management, timers, and security. To support only a DCE client, less functionality is required, and even MS-DOS is sufficient.

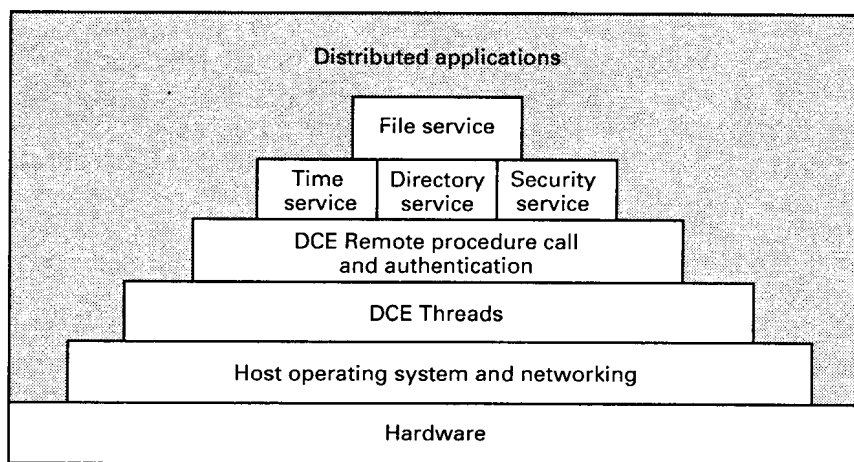


Fig. 10-1. Rough sketch of how the parts of DCE fit together.

On top of the operating system is the DCE threads package. If the operating system has a suitable threads package itself, the DCE threads library just serves to convert the interface to the DCE standard. If there is no native threads package, DCE supplies a threads package that runs almost entirely in user space, except for a few hundred lines of assembly code in the kernel for managing thread stacks. Next comes the RPC package, which, like the threads code is a collection of library procedures. A portion of the security is logically also in this layer, since it is required for performing authenticated RPC.

On top of the RPC layer come the various DCE services. Not every service runs on every machine. The system administrator determines which service to run where. The standard services are the time, directory, and security, with the distributed file service on top of them, as shown. In a typical configuration, these services run only on “system server” machines and not on client workstations. A short introduction to each of these follows.

The general function of the threads and RPC packages should be clear, but a short explanation of the services may be helpful now. Detailed discussions of

all the facilities and services will be presented later. The **distributed time service** is needed because each machine in the system normally has its own clock, so the concept of what time it is is more complicated than in a single system. For example, the UNIX *make* program examines the creation times of the source and binary files of a large program to determine which sources have changed since the binary was last made. Only these sources need to be recompiled. Consider, however, what happens if the sources and binaries are stored on different machines, and the clocks are not synchronized. The fact that the creation time for a binary file is later than the creation time for the corresponding source file does not mean that recompilation can be skipped. The difference might be a consequence of the two clocks being different. The DCE time service attempts to solve this problem by keeping the clocks synchronized in order to provide a global notion of time.

The **directory service** is used to keep track of the location of all resources in the system. These resources include machines, printers, servers, data, and much more, and they may be distributed geographically over the entire world. The directory service allows a process to ask for a resource and not have to be concerned about where it is, unless the process cares.

The **security service** allows resources of all kinds to be protected, so access can be restricted to authorized persons. For example, in a hospital information system, it might be the policy that a doctor could see all the medical information about her own patients, but not about other doctors' patients. People working in the billing department might be allowed to see all patients' records, but only the financial aspects. If a blood test is performed, the doctor can see what the medical results are and the bookkeeper can see how much it cost (but not the medical results). The security service provides tools that aid the construction of applications like this.

Finally, the **distributed file service** is a worldwide file system that provides a transparent way of accessing any file in the system in the same way. It can either be built on top of the hosts' native file systems or be used instead of them.

At the top of the DCE food chain are the distributed applications. These can use (or bypass) any of the DCE facilities and services. Simple applications may implicitly use RPC (via the library, and without realizing it) and little else, whereas more complex applications may make many explicit calls to all the services.

The picture given in Fig. 10-1 is not completely accurate, but it is hard to depict the dependencies between the parts because they are recursive. For example, the directory service uses RPC for internal communication among its various servers, but the RPC package uses the directory service to locate the destination. Thus it is not strictly true that the directory service layer is on top of RPC, as shown. As a second example, illustrating horizontal dependencies, the time service uses the security service to see who may set the clock, but the

security service uses the time service to issue permission tickets with short lifetimes. Nevertheless, to a first approximation, Fig. 10-1 gives an indication of the gross structure, so we will use it as our model henceforth.

10.1.4. Cells

Users, machines, and other resources in a DCE system are grouped together to form **cells**. Naming, security, administration, and other aspects of DCE are based upon these cells. Cell boundaries usually mirror organizational units, for example, a small company or a department of a large company might be one cell.

When determining how to group machines into cells, four factors should be taken into consideration:

1. Purpose.
2. Security.
3. Overhead.
4. Administration.

The first factor, purpose, means that the machines in a cell (and their users) should all be working toward a common goal or on a common long-term project (probably measured in years). The users should know each other and have more contact with each other than with people outside the cell. Departments in companies are often structured this way. Cells can also be organized around a common service offered, such as online banking, with all the automated teller machines and the central computer belonging to a single cell.

The second factor, security, has to do with the fact that DCE works better when the users in a cell trust each other more than they trust people outside the cell. The reason for this is that cell boundaries act like firewalls—getting at internal resources is straightforward, but accessing anything in another cell requires the two cells to perform an arms' length negotiation to make sure that they trust one another.

The third factor, overhead, is important because some DCE functions are optimized for intracell operation (e.g., security). Geography can play a role here because putting distant users in the same cell means that they will often have to go over a wide-area network to communicate. If the wide-area network is slow and unreliable, extra overhead will be incurred to deal with these problems and compensate for them where possible.

Finally, every cell needs an administrator. If all the people and machines in a cell belong to the same department or project, there should be no problem appointing an administrator. However, if they belong to two widely separated

departments, each with its own czar, it may be harder to agree upon one person to administer the cell and make cell-wide decisions.

Subject to these constraints, it is desirable to have as few cells as possible to minimize the number of operations that cross cell boundaries. Also, if an intruder ever breaks into a cell and steals the security data base, new passwords will have to be established with every other cell. The more of them there are, the more work is involved.

To make the idea of cells clearer, let us consider two examples. The first is a large manufacturer of electrical equipment whose products range from aircraft engines to toasters. The second is a publisher whose books cover everything from art to zoos. Since the electrical manufacturer's products are so diverse, it may organize its cells around products, as shown in Fig. 10-2(a), with different cells for the toaster group and the jet engine group. Each cell would contain the design, manufacturing, and marketing people, on the grounds that people marketing jet engines need more contact with people manufacturing jet engines than they do with people marketing toasters.

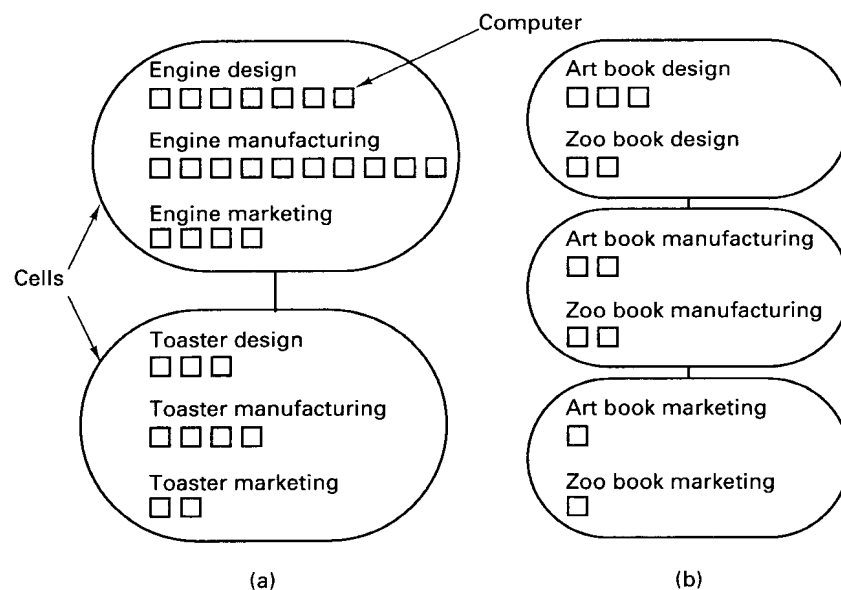


Fig. 10-2. (a) Two cells organized by product. (b) Three cells organized by function.

In contrast, the publisher would probably organize the cells as in Fig. 10-2(b) because manufacturing (i.e., printing and binding) a book on art is pretty much like manufacturing a book on zoos, so the differences between the departments are probably more significant than the differences between the products.

On the other hand, if the publisher has separate, autonomous divisions for childrens' books, trade books, and textbooks, all of which were originally separate companies with their own management structures and corporate cultures, arranging the cells by division rather than function may be best.

The point of these examples is that the determination of cell boundaries tends to be driven by business considerations, not technical properties of DCE.

Cell size can vary considerably, but all cells must contain a time server, a directory server, and a security server. It is possible for the time server, directory server, and security server all to run on the same machine. In addition, most cells will either contain one or more application clients (for doing work) or one or more application servers (for delivering a service). In a large cell, there might be multiple instances of the time, directory, and security servers, as well as hundreds of applications client and servers.

10.2. THREADS

The threads package, along with the RPC package, is one of the fundamental building blocks of DCE. In this section we will discuss the DCE threads package, focusing on scheduling, synchronization, and the calls available.

10.2.1. Introduction to DCE Threads

The DCE threads package is based on the P1003.4a POSIX standard. It is a collection of user-level library procedures that allow processes to create, delete, and manipulate threads. However, if the host system comes with a (kernel) threads package, the vendor can set up DCE to use it. The basic calls are for creating and deleting threads, waiting for a thread, and synchronizing computation between threads. Many other calls are provided for handling all the details and other functions.

A thread can be in one of four states, as shown in Fig. 10-3. A **running** thread is one that is actively using the CPU to do computation. A **ready** thread is willing and able to run, but cannot run because the CPU is currently busy running another thread. In contrast, a **waiting** thread is one that logically cannot run because it is waiting for some event to happen (e.g., a mutex to be unlocked). Finally, a **terminated** thread is one that has exited but has not yet been deleted and whose memory (i.e., stack space) has not yet been recovered. Only when another thread explicitly deletes it does a thread vanish.

On a machine with one CPU, only one thread can be actually running at a given instant. On a multiprocessor, several threads within a single process can be running at once, on different CPUs (true parallelism).

The threads package has been designed to minimize the impact on existing

State	Description
Running	The thread is actively using the CPU
Ready	The thread wants to run
Waiting	The thread is blocked waiting for some event
Terminated	The thread has exited but not yet been destroyed

Fig. 10-3. A thread can be in one of four states.

software, so programs designed for a single-threaded environment can be converted to multithreaded processes with a minimum of work. Ideally, a single-threaded program can be converted into a multithreaded one just by setting a parameter indicating that more threads should be used. However, problems can arise in three areas.

The first problem relates to signals. Signals can be left in their default state, ignored, or caught. Some signals are synchronous, caused specifically by the running thread. These include floating-point exception, causing a memory protection violation, and having your own timer go off. Others are asynchronous, caused by some external agency, such as the user hitting the DEL key to interrupt the running process.

When a synchronous signal occurs, it is handled by the current thread, except that if it is neither ignored nor caught, the entire process is killed. When an asynchronous signal occurs, the threads package checks to see if any threads are waiting for it. If so, it is passed to all the threads that want to handle it.

The second problem relates to the standard library, most of whose procedures are not reentrant. It can happen that a thread is busy allocating memory when a clock interrupt forces a thread switch. At the moment of the switch, the memory allocator's internal data structures may be inconsistent, which will cause problems if the newly scheduled thread tries to allocate some memory.

This problem is solved by providing jackets around some of the library procedures (mostly I/O procedures) that provide mutual exclusion for the individual calls. For the other procedures, a single global mutex makes sure that only one thread at a time is active in the library. The library procedures, such as *read* and *fork* are all jacketed procedures that handle the mutual exclusion and then call another (hidden) procedure to do the work. This solution is something of a quick hack. A better solution would be to provide a proper reentrant library.

The third problem has to do with the fact that UNIX system calls return their error status in a global variable, *errno*. If one thread makes a system call but just after the call completes, another thread is scheduled and it, too, makes a

system call, the original value of *errno* will be lost. A solution is provided by providing an alternative error handling interface. It consists of a macro that allows the programmer to inspect a thread-specific version of *errno* that is saved and restored upon thread switches. This solution avoids the need to examine the global version of *errno*. In addition, it is also possible to have system calls indicate errors by raising exceptions, thus bypassing the problem altogether.

10.2.2. Scheduling

Thread scheduling is similar to process scheduling, except that it is visible to the application. The scheduling algorithm determines how long a thread may run, and which thread runs next. Just as with process scheduling, many thread scheduling algorithms are possible.

Threads in DCE have priorities and these are respected by the scheduling algorithm. High-priority threads are assumed to be more important than low-priority threads, and therefore should get better treatment, meaning they run first and get a larger portion of the CPU.

DCE supports the three thread scheduling algorithms illustrated in Fig. 10-4. The first, FIFO, searches the priority queues from highest to lowest, to locate the highest priority queue with one or more threads on it. The first thread on this queue is then run until it finishes, either by blocking or exiting. In principle, the selected thread can run as long as it needs to. When it has finished, it is removed from the queue of runnable threads. Then the scheduler once again searches the queues from high to low and takes the first thread it finds.

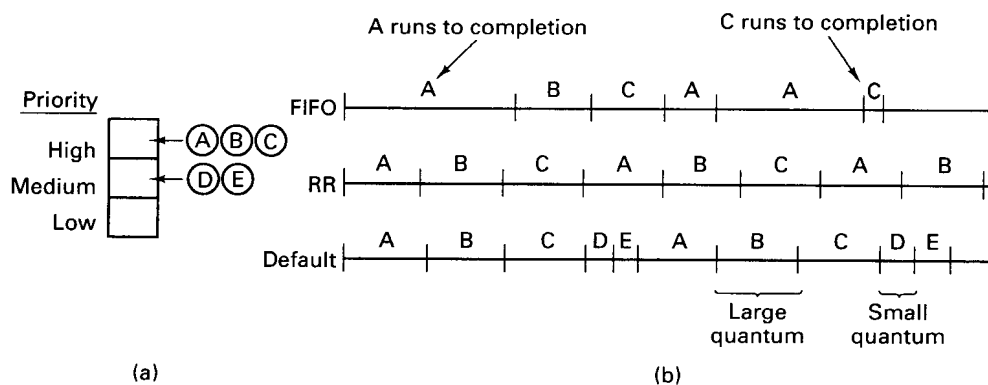


Fig. 10-4. (a) A system with five threads and three priority levels. (b) Three thread scheduling algorithms.

The second algorithm is round robin. Here the scheduler locates the highest populated queue and runs each thread for a fixed quantum. If a thread blocks or

exits before its quantum is up, it is (temporarily) removed from the queue system. If it uses up its entire quantum, it is suspended and placed at the end of its queue. In the middle example of Fig. 10-4(b), the threads *A*, *B*, and *C* will run alternately forever if they want to. The medium-priority threads *D* and *E* will never get a chance as long as one of the high-priority threads wants to run.

The third algorithm is the default algorithm. It runs the threads on all the queues using a time-sliced round-robin algorithm, but the higher the priority, the larger the quantum a thread gets. In this manner, all threads get to run and there is no starvation as in the second algorithm.

There is also a fourth algorithm that has variable-sized quanta, but with starvation. However, this one is not defined by POSIX, so it is not portable and should be avoided.

10.2.3. Synchronization

DCE provides two ways for threads to synchronize: mutexes and condition variables. Mutexes are used when it is essential to prevent multiple threads from accessing the same resource at the same time. For example, when moving items around on a linked list, partway through the move, the list will be in an inconsistent state. To prevent disaster, when one thread is manipulating the list, all other threads must be kept away. By requiring a thread to first successfully lock the mutex associated with the list before touching the list (and unlock it afterward), correct operation can be ensured.

Three kinds of mutexes are available, as shown in Fig. 10-5. They differ in the way they deal with nested locks. A **fast mutex** is analogous to a lock in a data base system. If a process tries to lock an unlocked record, it will succeed. However, if it tries to acquire the same lock a second time, it will block, waiting for the lock to be released, something that will never happen. Deadlock will occur.

Mutex type	Properties
Fast	Locking it a second time causes a deadlock
Recursive	Locking it a second time is allowed
Nonrecursive	Locking it a second time gives an error

Fig. 10-5. Three kinds of mutexes supported by DCE.

A **recursive mutex** allows a thread to lock a mutex that it has already locked. The idea is this. Suppose that the main program of a thread locks a mutex, then calls a procedure that also locks the mutex. To avoid deadlock, the

second lock is accepted. As long as the mutex is ultimately unlocked as many times as it is locked, the nesting can be arbitrarily deep. Although recursive mutexes are more user friendly, they are also considerably slower, so the programmer has to make a choice. As a compromise, DCE provides a third kind of mutex, one in which an attempt to lock a mutex that is already locked does not deadlock, but returns an error instead.

Condition variables provide a second synchronization mechanism. These are used in conjunction with mutexes. Typically, when a thread needs some resource, it uses a mutex to gain exclusive access to a data structure that keeps track of the status of the resource. If the resource is not available, the thread waits on a condition variable, which atomically suspends the thread and releases the mutex. Later, when another thread signals the condition variable, the waiting thread is restarted.

10.2.4. Thread Calls

The DCE threads package has a total of 54 primitives (library procedures). Many of these are not strictly necessary but are provided for convenience only. This approach is somewhat analogous to a four-function pocket calculator that has keys not only for +, −, ×, and /, but also has keys for +1, −1, × 2, × 10, × π , /2, and /10, on the grounds that these save the user time and effort. Due to the large number of calls, we will discuss only the most important ones (about half the total). Nevertheless, our treatment should give a reasonable impression of the available functionality.

Call	Description
Create	Create a new thread
Exit	Called by a thread when it is finished
Join	Like the WAIT system call in UNIX
Detach	Make it unnecessary for parent thread to wait when caller exits

Fig. 10-6. Selected DCE thread calls for managing threads. All the calls in this section are actually prefixed by *pthread_* (i.e., *pthread_create*, not *create*), which we have omitted to save space.

For our discussion, it is convenient to group the calls into seven categories, each dealing with a different aspect of threads and their use. The first category, listed in Fig. 10-6, deals with thread management. These calls allow threads to be created and for them to exit when done. A parent thread can wait for a child using *join*, which is similar to the WAIT system call in UNIX. If a parent has no

interest in a child and does not plan to wait for it, the parent can disown the child by calling *detach*. In this case, when the child thread exits, its storage is reclaimed immediately instead of having it wait for the parent to call *join*.

The DCE package allows the user to create, destroy, and manage **templates** for threads, mutexes, and condition variables. The templates can be set up to have appropriate initial values. When an object is created, one of the parameters to the create call is a pointer to a template. For example, a thread template can be created and given the attribute (property) that the stack size is 4K. Whenever a thread is created with that template as parameter, it will get an 4K stack. The point of having templates is to eliminate the need for specifying all the options as separate parameters. As the package evolves, the create calls can remain the same. Instead, new attributes can be added to the templates. Some of the template calls are listed in Fig. 10-7.

Call	Description
Attr_create	Create template for setting thread parameters
Attr_delete	Delete template for threads
Attr_setprio	Set the default scheduling priority in the template
Attr_getprio	Read the default scheduling priority from the template
Attr_setstacksize	Set the default stack size in the template
Attr_getstacksize	Read the default stack size from the template
Attr_mutexattr_create	Create template for mutex parameters
Attr_mutexattr_delete	Delete template for mutexes
Attr_mutexattr_setkind_np	Set the default mutex type in the template
Attr_mutexattr_getkind_np	Read the default mutex type from the template
Attr_condattr_create	Create template for condition variable parameters
Attr_condattr_delete	Delete template for condition variables

Fig. 10-7. Selected template calls.

The *attr_create* and *attr_delete* calls create and delete thread templates, respectively. Other calls allow programs to read and write the template's attributes, such as the stack size and scheduling parameters to be used for threads

created with the template. Similarly, calls are provided to create and delete templates for mutexes and condition variables. The need for the latter is not entirely obvious, since they have no attributes and no operations. Perhaps, the designers were hoping that someone would one day think of an attribute.

The third group deals with mutexes, which can be created and destroyed dynamically. Three operations are defined on mutexes, as shown in Fig. 10-8. The operations are for locking, unlocking mutexes, and for trying but accepting failure if locking cannot be done.

Call	Description
Mutex_init	Create a mutex
Mutex_destroy	Delete a mutex
Mutex_lock	Try to lock a mutex; if it is already locked, block
Mutex_trylock	Try to lock a mutex; fail if it is already locked
Mutex_unlock	Unlock a mutex

Fig. 10-8. Selected mutex calls.

Next come the calls relating to condition variables, listed in Fig. 10-9. Condition variables, too, can be created and destroyed dynamically. Threads can sleep on condition variables pending the availability of some needed resource. Two wakeup operations are provided: signaling, which wakes up exactly one waiting thread, and broadcasting, which wakes them all up.

Call	Description
Cond_init	Create a condition variable
Cond_destroy	Delete a condition variable
Cond_wait	Wait on a condition variable until a signal or broadcast arrives
Cond_signal	Wake up at most one thread waiting on the condition variable
Cond_broadcast	Wake up all the threads waiting on the condition variable

Fig. 10-9. Selected condition variable calls.

Figure 10-10 lists the three calls for manipulating per-thread global variables. These are variables that may be used by any procedure in the thread that

created them, but which are invisible to other threads. The concept of a per-thread global variable is not supported by any of the popular programming languages, so they have to be managed at run time. The first call creates an identifier and allocates storage, the second assigns a pointer to a per-thread global variable, and the third allows the thread to read back a per-thread global variable value. Many computer scientists consider global variables to be in the same league as that all-time great pariah, the GOTO statement, so they would no doubt rejoice at the idea of making them cumbersome to use. (The author once tried to design a programming language with a

`IKNOWTHISISASTUPIDTHINGTODOBUTNEVERTHELESSGOTO LABEL;`

statement, but was forcibly restrained from doing so by his colleagues.) It can be argued that having per-thread global variables use procedure calls instead of language scoping rules, like locals and globals, is an emergency measure introduced simply because most programming languages do not allow the concept to be expressed syntactically.

Call	Description
Keycreate	Create a global variable for this thread
Setspecific	Assign a pointer value to a per-thread global variable
Getspecific	Read a pointer value from a per-thread global variable

Fig. 10-10. Selected per-thread global variable calls.

The next group of calls (see Fig. 10-11) deals with killing threads and the threads' ability to resist. The *cancel* call tries to kill a thread, but sometimes killing a thread can have devastating effects, for example, if the thread has a mutex locked at the time. For this reason, threads can arrange for attempts to kill them to be enabled or disabled in various ways, very roughly analogous to the ability of UNIX processes to catch or ignore signals instead of being terminated by them.

Call	Description
Cancel	Try to kill another thread
Setcancel	Enable or disable ability of other threads to kill this thread

Fig. 10-11. Selected calls relating to killing threads.

Finally, our last group (see Fig. 10-12) is concerned with scheduling. The package allows the threads in a process to be scheduled according to FIFO,

round robin, preemptive, nonpreemptive, and other algorithms. By using these calls, the algorithm and priorities can be set. The system works best if threads do not elect to be scheduled with conflicting algorithms.

Call	Description
Setscheduler	Set the scheduling algorithm
Getscheduler	Read the current scheduling algorithm
Setprio	Set the scheduling priority
Getprio	Get the current scheduling priority

Fig. 10-12. Selected scheduling calls.

10.3. REMOTE PROCEDURE CALL

DCE is based on the client/server model. Clients request services by making remote procedure calls to distant servers. In this section we will describe how this mechanism appears to both sides and how it is implemented.

10.3.1. Goals of DCE RPC

The goals of the DCE RPC system are relatively traditional. First and foremost, the RPC system makes it possible for a client to access a remote service by simply calling a local procedure. This interface makes it possible for client (i.e., application) programs to be written in a simple way, familiar to most programmers. It also makes it easy to have large volumes of existing code run in a distributed environment with few, if any, changes.

It is up to the RPC system to hide all the details from the clients, and, to some extent, from the servers as well. To start with, the RPC system can automatically locate the correct server and bind to it, without the client having to be aware that this is occurring. It can also handle the message transport in both directions, fragmenting and reassembling them as needed (e.g., if one of the parameters is a large array). Finally, the RPC system can automatically handle data type conversions between the client and the server, even if they run on different architectures and have a different byte ordering.

As a consequence of the RPC system's ability to hide the details, clients and servers are highly independent of one another. A client can be written in C and a server in FORTRAN, or vice versa. A client and server can run on different

hardware platforms and use different operating systems. A variety of network protocols and data representations are also supported, all without any intervention from the client or server.

10.3.2. Writing a Client and a Server

The DCE RPC system consists of a number of components, including languages, libraries, daemons and utility programs, among others. Together these make it possible to write clients and servers. In this section we will describe the pieces and how they fit together. The entire process of writing and using an RPC client and server is summarized in Fig. 10-13.

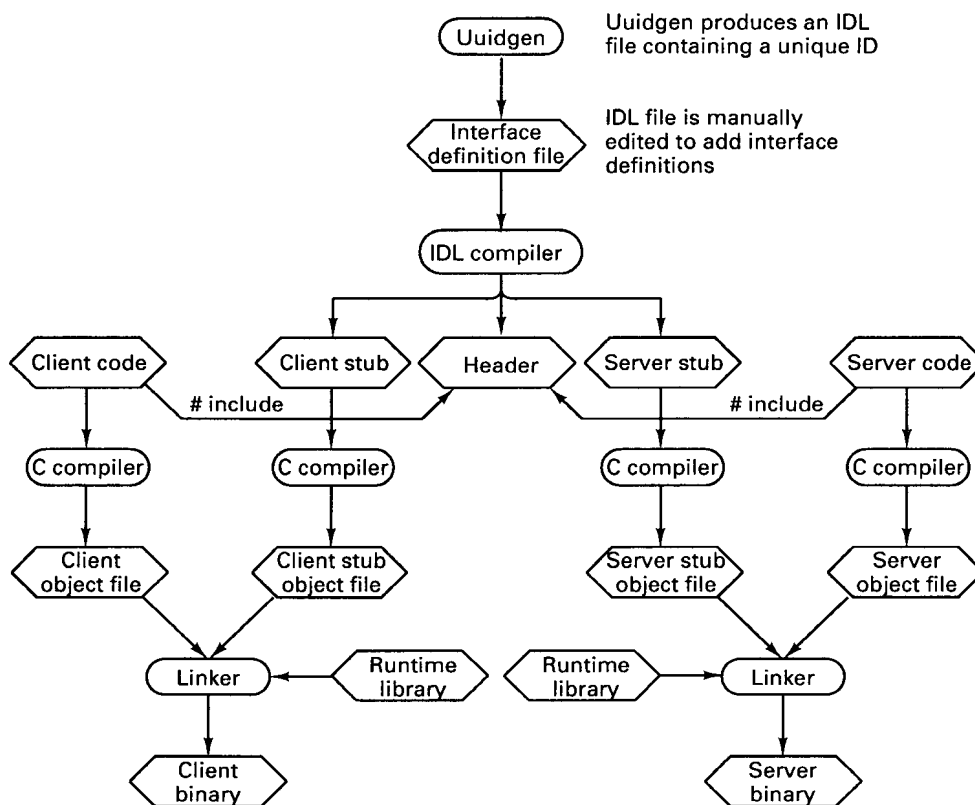


Fig. 10-13. The steps in writing a client and a server.

In a client/server system, the glue that holds everything together is the interface definition. This is effectively a contract between the server and its clients, specifying the services that the server is offering to the clients.

The concrete representation of this contract is a file, the interface definition file, which lists all the procedures that the server allows its clients to invoke remotely. Each procedure has a list of the names and types of its parameters and of its result. Ideally, the interface definition should also contain a formal definition of what the procedures do, but such a definition is beyond the current state of the art, so the interface definition just defines the syntax of the calls, not their semantics. At best the writer can add a few comments describing what he hopes the procedures will do.

Interface definitions are written in a language brilliantly named the **Interface Definition Language**, or **IDL**. It permits procedure declarations in a form closely resembling function prototypes in ANSI C. IDL files can also contain type definitions, constant declarations, and other information needed to correctly marshal parameters and unmarshal results.

A crucial element in every IDL file is an identifier that uniquely identifies the interface. The client sends this identifier in the first RPC message and the server verifies that it is correct. In this way, if a client inadvertently tries to bind to the wrong server, or even to an older version of the right server, the server will detect the error and the binding will not take place.

Interface definitions and unique identifiers are closely related in DCE. As illustrated in Fig. 10-13, the first step in writing a client/server application is usually calling the *uuidgen* program, asking it to generate a prototype IDL file containing an interface identifier guaranteed never to be used again in any interface generated anywhere by *uuidgen*. Uniqueness is ensured by encoding in it the location and time of creation. It consists of a 128-bit binary number represented in the IDL file as an ASCII string in hexadecimal.

The next step is editing the IDL file, filling in the names of the remote procedures and their parameters. It is worth noting that RPC is not totally transparent—for example, the client and server cannot share global variables—but the IDL rules make it impossible to express constructs that are not supported.

When the IDL file is complete, the IDL compiler is called to process it. The output of the IDL compiler consists of three files:

1. A header file (e.g., *interface.h*, in C terms).
2. The client stub.
3. The server stub.

The header file contains the unique identifier, type definitions, constant definitions, and function prototypes. It should be included (using *#include*) in both the client and server code.

The client stub contains the actual procedures that the client program will call. These procedures are responsible for collecting and packing the parameters

into the outgoing message and then calling the runtime system to send it. The client stub also handles unpacking the reply and returning values to the client.

The server stub contains the procedures called by the runtime system on the server machine when an incoming message arrives. These, in turn, call the actual server procedures that do the work.

The next step is for the application writer to write the client and server code. Both of these are then compiled, as are the two stub procedures. The resulting client code and client stub object files are then linked with the runtime library to produce the executable binary for the client. Similarly, the server code and server stub are compiled and linked to produce the server's binary. At runtime, the client and server are started to make the application run.

10.3.3. Binding a Client to a Server

Before a client can call a server, it has to locate the server and bind to it. Naive users can ignore the binding process and let the stubs take care of it automatically, but binding happens nevertheless. Sophisticated users can control it in detail, for example, to select a specific server in a particular distant cell. In this section we will describe how binding works in DCE.

The main problem in binding is how the client locates the correct server. In theory, broadcasting a message containing the unique identifier to every process in every cell and asking all servers for it to please raise their hands might work (security considerations aside), but this approach is so slow and wasteful that it is not practical. Besides, not all networks support broadcasting.

Instead, server location is done in two steps:

1. Locate the server's machine.
2. Locate the correct process on that machine.

Different mechanisms are used for each of these steps. The need to locate the server's machine is obvious, but the problem with locating the server once the machine is known is more subtle. Basically, what it comes down to is that for a client to communicate reliably and securely with a server, a network connection is generally required. Such a connection needs an **endpoint**, a numerical address on the server's machine to which network connections can be attached and messages sent. Having the server choose a permanent numerical address is risky, since another server on the same machine might accidentally choose the same one. For this reason, endpoints can be dynamically assigned, and a database of (server, endpoint) entries is maintained on each server machine by a process called the **RPC daemon**, as described below.

The steps involved in binding are shown in Fig. 10-14. Before it becomes available for incoming requests, the server must ask the operating system for an

endpoint. It then registers this endpoint with the RPC daemon. The RPC daemon records this information (including which protocols the server speaks) in the endpoint table for future use. The server also registers with some cell directory server, passing it the number of its host.

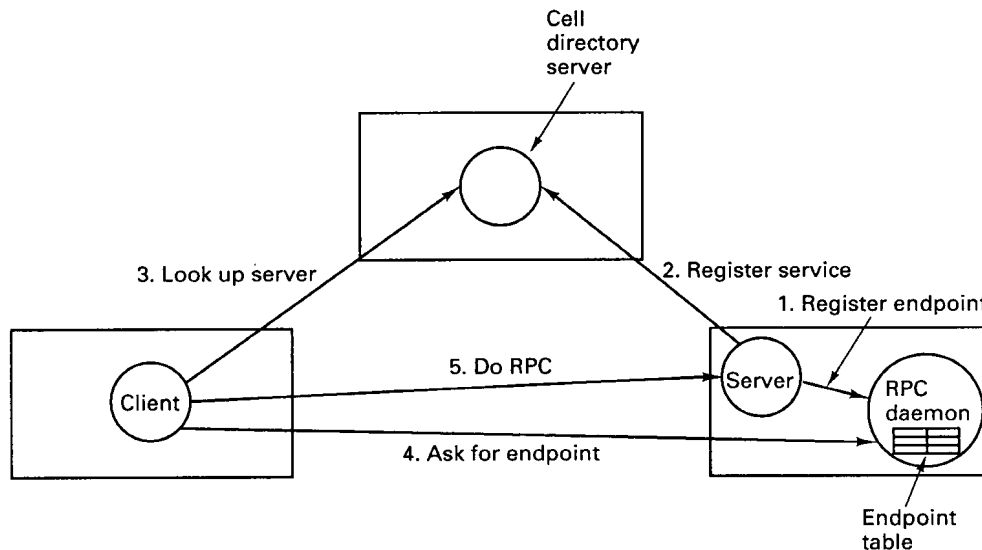


Fig. 10-14. Client-to-server binding in DCE.

Now let us look at the client side. In the simplest case, at the time of the first RPC, the client stub asks the cell directory server to find it a host running an instance of the server. The client then goes to the RPC daemon, which has a well-known endpoint, and asks it to look up the endpoint (e.g., TCP port) in its endpoint table. Armed with this information, the RPC can now take place. On subsequent RPCs this lookup is not needed. DCE also gives clients the ability to do more sophisticated searches for a suitable server when that is needed. Authenticated RPC is also an option. We will discuss authentication and protection later in this chapter.

10.3.4. Performing an RPC

The actual RPC is carried out transparently and in the usual way. The client stub marshals the parameters and passes the resulting buffer (possibly in chunks) to the runtime library for transmission using the protocol chosen at binding time. When a message arrives at the server side, it is routed to the correct server based on the endpoint contained in the incoming message. The runtime library passes

the message to the server stub, which unmarshals the parameters and calls the server. The reply goes back by the reverse route.

DCE provides several semantic options. The default is at-most-once operation, in which case no call is ever carried out more than once, even in the face of system crashes. In practice, what this means is that if a server crashes during an RPC and then recovers quickly, the client does not repeat the operation, for fear that it might already have been carried out once.

Alternatively, it is possible to mark a remote procedure as idempotent (in the IDL file), in which case it can be repeated multiple times without harm. For example, reading a specified block from a file can be tried over and over until it succeeds. When an idempotent RPC fails due to a server crash, the client can wait until the server reboots and then try again. Other semantics are also theoretically available (but rarely used), including broadcasting the RPC to all the machines on the local network.

10.4. TIME SERVICE

Time is an important concept in most distributed systems. To see why, consider a research program in radio astronomy. A number of radio telescopes spread all over the world observe the same celestial radio source simultaneously, accurately recording the data and the observation time. The data are sent over a network to a central computer for processing. For some kinds of experiments (e.g., long-baseline interferometry), it is essential for the analysis that the various data streams be synchronized exactly. Thus the experiment succeeds or fails with the ability to synchronize remote clocks accurately.

As another example, in a computerized stock trading system, it might matter who bid first on a block of stock offered for sale. If the bidders were located in New York, San Francisco, London, and Tokyo, timestamping the bids might be one way to achieve fairness. However, if the clocks at all these locations were not properly synchronized, the whole scheme would collapse and result in numerous lawsuits featuring expert witnesses trying to explain the concept of the speed of light to bewildered juries.

To try to prevent problems like these, DCE has a service called **DTS (Distributed Time Service)**. The goal of DTS is to keep clocks on separate machines synchronized. Getting them synchronized once is not enough, because the crystals in different clocks tick at a slightly different rates, so the clocks gradually drift apart. For example, a clock might be rated to have a relative error of one part in a million. This means that even if it is set perfectly, after 1 hour, the clock could be off by as much as 3.6 msec either way. After 1 day it could be off by as much as 86 msec either way. After a month, two clocks that were synchronized precisely might now differ by 5 seconds.